

university of groningen / faculty of science and engineering / mathematics and applied mathematics

Randomized approaches to approximating solutions of large kernel systems for Kernel Ridge Regression using the Nyström Approximation

Bachelor's Project Mathematics

April 2026

Student: Yuanhu Charles Tao

Student number: 5350980

First supervisor: Dr. M. Meier

Second supervisor: Prof. J.G. Peypouquet

Abstract

Kernel ridge regression uses a kernel function to measure pairwise similarities between n data points, and these values form the $n \times n$ kernel matrix K. This leads to a linear system involving the matrix K. For large datasets, the main computational difficulty is the need to explicitly form the full matrix K and directly solve the system. This project therefore studies randomized Nyström approximations, where only $l \ll n$ columns of K are used to build a low rank approximation. Uniform sampling, approximate leverage score sampling and recursive ridge leverage score sampling were compared against a full KRR baseline. The final experiments used five independent column selections on the three datasets. Uniform sampling was the strongest practical baseline and performed similarly to the informed methods with almost no sampling cost. The required value of l varied strongly across datasets, but the experiments generally showed diminishing returns as l increased. These results suggest using uniform sampling as the default but no universal value of l can be recommended across datasets.

Acknowledgements

I would like to express my sincerest gratitude to my first supervisor, Dr. Maike Meier, for her guidance and support throughout this project. Our weekly meetings, together with the materials and detailed feedback she provided, were essential to the development of this thesis. I am especially grateful for her understanding during a difficult period with my health and for her support in arranging an extension, which gave me the opportunity to complete the project to the best of my ability. I also greatly appreciated her making time to attend both my practice presentation and final presentation while she was on leave.

I would also like to thank Prof. Juan Peypouquet for his valuable feedback during the final presentation, particularly his suggestion to repeat the randomized experiments and report their variability, which significantly strengthened the experimental results.

Contents

1 Introduction **4**

2 Kernel Ridge Regression **6**
2.1 Introduction to the regression problem 6
2.1.1 Kernel trick 6
2.1.2 Positive definite kernels and RKHS 7
2.1.3 Representer form and the kernel matrix 8
2.2 Computational cost of full KRR 10
2.3 Kernel choices 11
2.4 Singular values and low-rank structure 11
2.5 Runtime and accuracy experiments 12

3 Nyström Approximation **15**
3.1 General Nyström form 15
3.2 Randomized Nyström approximation 17
3.3 Nyström approximation and the KRR system 18
3.4 Column sampling 18
3.4.1 Uniform sampling 21
3.4.2 Leverage score sampling 23

4 Informed practical column sampling **25**
4.1 Approximate leverage score sampling 25
4.1.1 Comparison to exact leverage scores 26
4.1.2 Initial sample sizes and error 28
4.1.3 Limitations of approximate leverage sampling 28
4.2 Recursive ridge leverage score sampling 29
4.2.1 Ridge leverage scores and effective dimension 30
4.2.2 Recursive approximation of the scores 31
4.2.3 Approximation guarantee 33
4.2.4 Relation between the sampling parameter and KRR regularization . 33

5 Experimental results and analysis **36**
5.1 Final experimental setup and aggregation 36
5.2 Prediction error against sampled columns 37
5.3 Runtime and prediction trade off 38
5.4 Runtime analysis and solver behaviour 39
5.5 Secondary results and residual diagnostics 40
5.6 Repeated experiments on additional datasets 41

6 Conclusions **44**
6.1 Limitations and future work 45

1 Introduction

Kernel methods provide a way to capture nonlinear relationships by projecting data onto a higher-dimensional space and describing the data through pairwise similarities. Kernel functions are used to compute inner products in the higher-dimensional space without explicitly mapping the data (Hofmann et al., 2008). This allows flexible regression models to be constructed, but the resulting matrix computations become expensive as the number of observations increases (Bach, 2013; Alaoui and Mahoney, 2015). This project therefore studies randomized Nyström approximation, one of the most popular techniques that reduces this computational cost for kernel ridge regression while trying to preserve its prediction quality (Williams and Seeger, 2000; Musco and Musco, 2017).

Many regression problems start with data (x_i, y_i) for $i = 1, \ldots, n$, where n is the number of observations, $x_i \in \mathbb{R}^d$ are the inputs and $y_i \in \mathbb{R}$ are the outputs. We would like to use these data to create a model for the relationship between the inputs and outputs. One formulation of this task is kernel ridge regression, which uses a kernel function $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ to measure pairwise similarities between the inputs. These values form the $n \times n$ kernel matrix K, whose entries are $K_{ij} = k(x_i, x_j)$. The details of this formulation are introduced in Chapter 2.

For large datasets, the main computational difficulty is the need to explicitly form the full matrix K, which costs $O(n^2)$ time. This is the part that becomes infeasible first and takes the most time and resources. Solving the resulting system directly can also cost about $O(n^3)$ operations, so this is fine for small tests, but not realistic when n becomes large (Drineas et al., 2005; Bach, 2013; Alaoui and Mahoney, 2015).

Therefore in this project we investigate the Nyström approximation of the kernel matrix. Instead of forming all of K, we select an index set $I \subset \{1, \ldots, n\}$ containing $l \ll n$ column indices. The selected columns form the matrix $C = K_{:,I} \in \mathbb{R}^{n \times l}$, while their intersections with the corresponding rows form the smaller matrix $W = K_{I,I} \in \mathbb{R}^{l \times l}$. These two matrices are then used to construct a low-rank approximation $\tilde{K} \approx K$, for example $\tilde{K} = CW^{\dagger}C^T$. This is a randomized approach for kernel matrices (Williams and Seeger, 2000; Drineas et al., 2005; Kumar et al., 2012; Gittens and Mahoney, 2013, 2016). The point is to reduce the computational cost while still keeping enough information from the original kernel matrix. The full construction is introduced in Chapter 3.

Nyström needs a set of l columns to build $\tilde{K}$. The simplest choice is uniform random sampling, i.e. pick columns uniformly at random (Kumar et al., 2012; Gittens and Mahoney, 2016). Smarter strategies try to pick columns that are more informative so that fewer columns are needed for similar accuracy. One idea is to sample according to ridge leverage scores, which give higher probability to points that matter more for the kernel ridge regression solution (Alaoui and Mahoney, 2015). Another idea is to approximate these leverage type probabilities adaptively without ever constructing the whole K (Calandriello et al., 2017).

This project is therefore about how to decide which columns to sample and how many columns l are needed for a good Nyström approximation in the KRR setting. We will compare different sampling strategies, starting from uniform random sampling as a baseline, and then compare against smarter strategies motivated by the literature. The experiments will compare these strategies in terms of prediction accuracy and computational complexity as l changes.

The contribution of this project is an empirical comparison of three Nyström column sampling strategies under a common factorized KRR solver, together with an investigation of how the required columns count and accuracy/runtime trade off vary across datasets.

The research questions are:

- What Nyström column sampling strategies give the strongest performance for KRR in terms of prediction accuracy and runtime?
- How large does l need to be to obtain a good approximation and prediction accuracy, and how does this vary across datasets, sampling strategies and for recursive RLS, the parameter λ_{RLS}?
- What accuracy/runtime trade off arises as l increases, and what practical sampling approach can be recommended?

This thesis is organised as follows. Chapter 2 introduces kernel ridge regression and the kernel matrices used in the project. It also gives the first singular value experiments, which motivate why a low-rank approximation is a reasonable thing to try. Chapter 3 introduces the Nyström approximation, starting from the general positive semidefinite matrix formula and then moving to column sampling. Chapter 4 introduces approximate leverage score sampling and recursive ridge leverage score sampling. Chapter 5 presents the experimental comparison and analyses the prediction runtime trade off. Chapter 6 answers the research questions and discusses limitations and future work.

2 Kernel Ridge Regression

In this chapter we introduce the kernel ridge regression problem that lays the foundation for this project. First, the regression problem is written as an optimization problem over a reproducing kernel Hilbert space. Then the representer theorem is used to turn this function-space problem into a finite-dimensional problem involving the kernel matrix. Finally, we introduce the kernels used in the experiments and give some numerical checks that motivate using low-rank approximations.

2.1 Introduction to the regression problem

In this section we introduce the regression problem that will be used throughout the project. Assume we have a dataset (x_i, y_i) for $i = 1, \dots, n$, where $x_i \in \mathbb{R}^d$ are inputs and $y_i \in \mathbb{R}$ are outputs. We would like to use these data to create a model for the relationship between the inputs and outputs.

In ordinary linear regression, the model is searched for in a finite-dimensional linear function class. Kernel ridge regression instead searches over a reproducing kernel Hilbert space (RKHS) $\mathcal{H}$. Kernel ridge regression searches for a function $f \in \mathcal{H}$ that fits the training data while keeping the RKHS norm small. The optimization problem is

$$\min_{f \in \mathcal{H}} \|f(X) - y\|_2^2 + \lambda \|f\|_{\mathcal{H}}^2,$$

which is also

$$\min_{f \in \mathcal{H}} \sum_{i=1}^{n} (f(x_i) - y_i)^2 + \lambda \|f\|_{\mathcal{H}}^2.$$

Here $\lambda > 0$ is the regularization parameter. The first term measures the error on the training data, while the second term penalizes functions with large RKHS norm. In other words, λ controls the trade off between fitting the data and overfitting (Saunders et al., 1998; Hofmann et al., 2008).

The optimization problem above is still written over a function space, thus in the next sections we will define the RKHS and discuss its connection to kernels.

2.1.1 Kernel trick

Suppose the data matrix is

$$X \in \mathbb{R}^{n \times d},$$

where each row is an input point $x_i \in \mathbb{R}^d$. Let $\mathcal{X} \subseteq \mathbb{R}^d$ denote the input space and let $\mathcal{F}$ denote a feature Hilbert space. A kernel method can be viewed as first mapping the data onto a feature space by a map

$$\phi : \mathcal{X} \to \mathcal{F}.$$

If the feature vectors were written as rows of a matrix, this would give an implicit feature matrix

$$\Phi \in \mathbb{R}^{n \times d'},$$

where d' may be much larger than d or even infinite. The associated kernel function is

$$k(x, z) = \langle \phi(x), \phi(z) \rangle_{\mathcal{F}}.$$

And the corresponding kernel matrix is

$$K = \Phi \Phi^T \in \mathbb{R}^{n \times n},$$

with entries

$$K_{ij} = k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle_{\mathcal{F}}.$$

The kernel trick is that we do not need to explicitly construct $\phi(x_i)$ or the matrix Φ. Instead, we compute the inner products in the feature space directly through the kernel function $k(x_i, x_j)$ (Hofmann et al., 2008).

2.1.2 Positive definite kernels and RKHS

Before defining the RKHS, we first need to specify what kind of kernel functions can be used. The reason is that the kernel will later be used to define an inner product between functions, so it must satisfy the basic properties needed for an inner product to be valid.

Definition 2.1 (Positive definite kernel; Aronszajn, 1950; Hofmann et al., 2008)**.** Let $\mathcal{X} \subseteq \mathbb{R}^d$ be the input space. A symmetric function

$$k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$$

is called a positive definite kernel if, for any points $x_1, \ldots, x_n \in \mathcal{X}$ and any coefficients $\alpha_1, \ldots, \alpha_n \in \mathbb{R}$,

$$\sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j k(x_i, x_j) \geq 0.$$

The symmetry condition means that $k(x_i, x_j) = k(x_j, x_i)$, which matches the symmetry of an inner product. The positive definiteness condition will be used below to make sure that the squared norm induced by the kernel-defined inner product is nonnegative.

The value $k(x_i, x_j)$ measures the similarity between two inputs x_i and x_j according to the chosen kernel. If one input is fixed then $k(\cdot, x_i)$ is a function of the remaining argument. This is called a kernel section centered at x_i.

The RKHS associated with k is built from these kernel sections. First consider finite linear combinations of kernel sections with arbitrary centers in the input space, not only the training points.

$$f(\cdot) = \sum_{i=1}^{n} \alpha_i k(\cdot, x_i), \qquad x_i \in \mathcal{X}.$$

On this space, the inner product is first defined on individual kernel sections by

$$\langle k(\cdot, x_i), k(\cdot, x_j) \rangle_{\mathcal{H}} = k(x_i, x_j).$$

The inner product rule is then extended linearly from individual kernel sections to finite linear combinations. Therefore, for two finite linear combinations

$$f(\cdot) = \sum_{i=1}^{m} \alpha_i k(\cdot, x_i), \qquad g(\cdot) = \sum_{j=1}^{r} \beta_j k(\cdot, y_j),$$

the inner product is

$$\langle f, g \rangle_{\mathcal{H}} = \sum_{i=1}^{m} \sum_{j=1}^{r} \alpha_i \beta_j k(x_i, y_j).$$

In particular, when $g = f$, this gives the RKHS norm. We have

$$\|f\|_{\mathcal{H}}^2 = \left\langle \sum_{i=1}^{n} \alpha_i k(\cdot, x_i), \sum_{j=1}^{n} \alpha_j k(\cdot, x_j) \right\rangle_{\mathcal{H}} = \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j \langle k(\cdot, x_i), k(\cdot, x_j) \rangle_{\mathcal{H}}.$$

by the bilinearity of the inner product.

To ensure that the squared norm is nonnegative, we therefore need the kernel to be positive definite. This is the pre-Hilbert space generated by the kernel sections, then we can obtain the RKHS by completing this space (Aronszajn, 1950; Hofmann et al., 2008; Zhu, 2023).

Definition 2.2 (Reproducing kernel Hilbert space; Aronszajn, 1950; Hofmann et al., 2008; Zhu, 2023)**.** Given a positive definite kernel k, the associated reproducing kernel Hilbert space $\mathcal{H}$ is the Hilbert space obtained by completing the span of the kernel sections:

$$\mathcal{H} = \overline{\operatorname{span}\{k(\cdot, x) : x \in \mathcal{X}\}}.$$

It has the reproducing property

$$f(x) = \langle f, k(\cdot, x)\rangle_{\mathcal{H}}$$

for every $f \in \mathcal{H}$ and every $x \in \mathcal{X}$.

The reproducing property means that evaluating f at a point can be written as an inner product with the kernel section centered at that point.

The RKHS norm $\|f\|_{\mathcal{H}}$ gives a way to penalize the size of the function in this space. In later sections, after the representer theorem shows that the KRR solution has the form

$$f(\cdot) = \sum_{i=1}^{n} \alpha_i k(\cdot, x_i),$$

the same norm formula becomes

$$\|f\|_{\mathcal{H}}^2 = \sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j \langle k(\cdot, x_i), k(\cdot, x_j)\rangle_{\mathcal{H}} = \sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j k(x_i, x_j) = \alpha^T K\alpha,$$

where we used the reproducing property in the second equality. Thus the positive definiteness condition will also imply that the kernel matrix K is positive semidefinite.

One reason this norm can be interpreted as controlling the variation of f is the Cauchy-Schwarz inequality together with the reproducing property. For two inputs x and x',

$$|f(x) - f(x')| = |\langle f, k(\cdot, x) - k(\cdot, x')\rangle_{\mathcal{H}}| \le \|f\|_{\mathcal{H}}\|k(\cdot, x) - k(\cdot, x')\|_{\mathcal{H}}.$$

Thus a smaller norm limits how much the function can vary relative to the distance induced by the kernel. This motivates the regularization term $\lambda\|f\|_{\mathcal{H}}^2$ in kernel ridge regression (Hofmann et al., 2008; Saunders et al., 1998; Zhu, 2023).

The KRR objective is now well defined, but we still have to deal with the infinite dimensionality of $\mathcal{H}$. We will discuss this in the next sections.

2.1.3 Representer form and the kernel matrix

The optimization problem above is written over the whole function space $\mathcal{H}$, so at first it is not clear how it can be solved computationally. The representer theorem gives the key reduction.

Theorem 2.1 (Representer theorem; Kimeldorf and Wahba, 1971; Schölkopf et al., 2001)**.** *Let $\{(x_i, y_i)\}_{i=1}^n$ be a training set, and let $\mathcal{H}$ be the RKHS associated with the kernel k. Consider an optimization problem of the form*

$$\hat{f} = \arg\min_{f\in\mathcal{H}} \mathcal{L}\big(\{y_i, f(x_i)\}_{i=1}^n\big) + p\big(\|f\|_{\mathcal{H}}^2\big),$$

where $\mathcal{L}$ is a loss function depending only on the fitted values at the training points and p is a monotone penalty function. Then the minimizer can be written as a finite linear combination of kernel functions centered at the training points:

$$\hat{f}(\cdot) = \sum_{i=1}^{n} \alpha_i k(\cdot, x_i),$$

for some coefficients $\alpha_1, \ldots, \alpha_n \in \mathbb{R}$.

For KRR, this theorem applies because the loss depends only on $f(x_1), \ldots, f(x_n)$ and the penalty is $\lambda \|f\|_{\mathcal{H}}^2$. The reason why this is useful is that we no longer need to search over all functions in $\mathcal{H}$. We have reduced the problem from infinite dimensional to finite dimensional, then we only need to determine the n coefficients $\alpha_1, \ldots, \alpha_n$. To see why this is possible, let

$$\mathcal{S} = \text{span}\{k(\cdot, x_1), \ldots, k(\cdot, x_n)\}.$$

Any function $f \in \mathcal{H}$ can be decomposed into a part inside this span and a part orthogonal to it, so

$$f = f_{\mathcal{S}} + h, \qquad f_{\mathcal{S}} \in \mathcal{S}, \qquad h \perp \mathcal{S}.$$

For each training point x_i, the reproducing property gives

$$h(x_i) = \langle h, k(\cdot, x_i) \rangle_{\mathcal{H}} = 0.$$

Therefore

$$f(x_i) = f_{\mathcal{S}}(x_i) + h(x_i) = f_{\mathcal{S}}(x_i),$$

so replacing f by $f_{\mathcal{S}}$ does not change any fitted value on the training data. This means

$$\sum_{i=1}^{n} (f(x_i) - y_i)^2 = \sum_{i=1}^{n} (f_{\mathcal{S}}(x_i) - y_i)^2.$$

At the same time,

$$\|f\|_{\mathcal{H}}^2 = \|f_{\mathcal{S}}\|_{\mathcal{H}}^2 + \|h\|_{\mathcal{H}}^2,$$

so the extra part can only increase the penalty term. Hence the minimizer can be taken inside the finite-dimensional span $\mathcal{S}$ (Schölkopf et al., 2001; Zhu, 2023).

To rewrite the objective in matrix form, we will define the kernel matrix

$$K \in \mathbb{R}^{n \times n}, \qquad K_{ij} = k(x_i, x_j).$$

Since k is symmetric and positive definite, the matrix K is symmetric positive semidefinite.

We now rewrite both terms in the KRR objective using α and K. For the training points we have

$$f(x_i) = \sum_{j=1}^{n} \alpha_j k(x_i, x_j),$$

so in vector form

$$f(X) = K\alpha.$$

Therefore the data fitting term becomes

$$\sum_{i=1}^{n} (f(x_i) - y_i)^2 = \|K\alpha - y\|_2^2.$$

The RKHS norm term can also be written using the kernel matrix as

$$\|f\|_{\mathcal{H}}^2 = \sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j k(x_i, x_j) = \sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i K_{ij} \alpha_j = \alpha^T K \alpha.$$

Substituting the data fitting term and the RKHS norm term into the objective gives the finite-dimensional problem

$$\min_{\alpha \in \mathbb{R}^n} \|K\alpha - y\|_2^2 + \lambda \alpha^T K \alpha.$$

Expand

$$\|K\alpha - y\|_2^2 = (K\alpha - y)^T (K\alpha - y),$$

and then differentiate

$$(K\alpha - y)^T (K\alpha - y) + \lambda \alpha^T K \alpha$$

with respect to α. Since K is symmetric this gives

$$2K(K\alpha - y) + 2\lambda K\alpha = 0.$$

This then reduces to

$$(K + \lambda I)\alpha = y.$$

Therefore the coefficient vector is

$$\alpha = (K + \lambda I)^{-1} y.$$

The prediction for a new point x is given by

$$\hat{f}(x) = \sum_{j=1}^{n} \alpha_j k(x, x_j).$$

Let x_i^{test} denote the ith test input and x_j^{train} denote the jth training input. And $K_{\text{test,train}}$ is the matrix with entries

$$(K_{\text{test,train}})_{ij} = k(x_i^{\text{test}}, x_j^{\text{train}}),$$

then the test predictions are

$$\hat{y}_{\text{test}} = K_{\text{test,train}} \alpha.$$

This completes the main transition in this chapter, the original problem over functions in the space $\mathcal{H}$ has been reduced to solving a finite-dimensional linear system with kernel matrix K (Saunders et al., 1998; Alaoui and Mahoney, 2015).

2.2 Computational cost of full KRR

The regularization term is important numerically. Since K can have very small eigenvalues or even zero eigenvalues, adding λI shifts all eigenvalues by λ so $K + \lambda I$ is positive definite. This makes the system uniquely solvable and improves the conditioning of the problem.

For large datasets, the main computational difficulty is that K is an $n \times n$ matrix. Forming the full kernel matrix requires $O(n^2)$ kernel evaluations and storing it requires $O(n^2)$ memory. Solving the system directly, for example by Cholesky decomposition, costs $O(n^3)$ operations. For small n this is fine but for large datasets this quickly becomes infeasible (Bach, 2013; Alaoui and Mahoney, 2015).

This is the computational reason for studying kernel approximation. The matrix formulation is finite-dimensional, but the dimension is the number of training observations n. Therefore when n is large it is natural to replace the full kernel matrix by a cheaper approximation $K \approx \widetilde{K}$.

The rest of this chapter studies the behaviour of the full kernel matrix and the full KRR solution, then we introduce the Nyström approximation in Chapter 3.

2.3 Kernel choices

In the experiments we start with three common kernels. The linear kernel is

$$k(x, z) = x^T z.$$

The polynomial kernel is

$$k(x, z) = (x^T z + 1)^2.$$

The Gaussian radial basis function (rbf) kernel is

$$k(x, z) = \exp(-\gamma \|x - z\|_2^2).$$

The parameter γ controls how local the Gaussian kernel is (Hofmann et al., 2008; Zhu, 2023). Larger γ makes points only strongly similar to very nearby points while smaller γ makes the similarities decay more slowly.

These choices give a range of model flexibility. The linear kernel is the simplest option and is used to capture linear structure in the data. The polynomial kernel can capture nonlinear feature interactions unlike the linear kernel. The RBF kernel is used as a more flexible nonlinear kernel, and its behaviour depends strongly on the choice of γ. Since this project is about KRR and Nyström approximation, these kernels are used mainly to compare different kernel matrices rather than to give a general rule for which kernel is best.

2.4 Singular values and low-rank structure

Given a large matrix, if many singular values are very small then much of the matrix can be approximated using a smaller rank (Eckart and Young, 1936). This is why in this section we look at singular values of K for real data. The point is to check whether a low-rank approximation is reasonable for real world data.

For this real data experiment we use the Superconductivity dataset from the UCI Machine Learning Repository (Hamidieh, 2018). This is a regression dataset with $n = 21263$ samples and $d = 81$ numerical input features, and the output is the critical temperature. This fits the KRR setting since the target variable is continuous.

This dataset is also large enough to show the computational problem. If the full dataset is used, the kernel matrix has size $n \times n$ where $n = 21,263$, this means there are $n^2 \approx 10^8$ entries in K which corresponds to about 3.6 GB in memory, not to mention the construction computational cost.

This is before solving the linear system or computing any decomposition. So the full matrix is already expensive, and this is the reason that an approximation method such as Nyström is relevant.

We now look at the singular values of three kernels linear, polynomial, and Gaussian RBF. The singular values are computed using the full kernel matrix, so this is just a check of the low-rank structure. The point is to see whether the singular values decay in practice for real data.

For the singular value plots, the singular values are normalised by the largest singular value, so the plotted quantity is

$$\frac{\sigma_i(K)}{\sigma_1(K)}.$$

This makes the three kernels easier to compare since all three curves start at 1. The plots are also stopped once the singular values reach around 10^{-16}, since after that the remaining tail is machine precision.

Figure 2.1 compares the three kernels used in this project. Figure 2.1(a) shows the linear kernel. Since $K = XX^T$ for a linear kernel and the data have 81 features, the rank is at most 81, so the singular values drop to zero after that point. Figure 2.1(b)

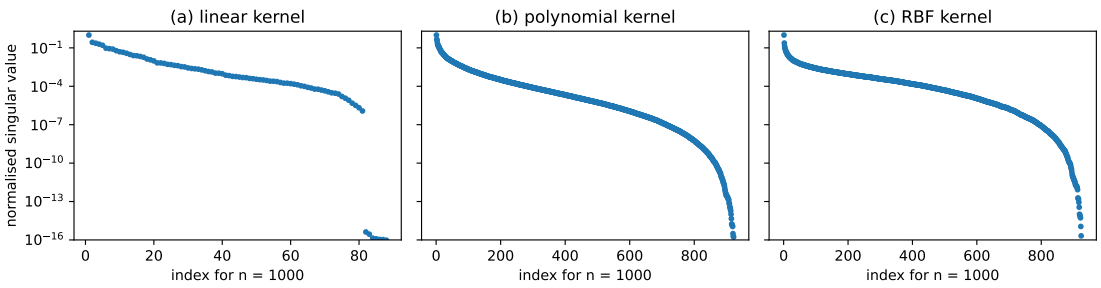


Figure 2.1: Normalised singular value decay for the three kernel matrices: (a) linear, (b) polynomial and (c) RBF.

shows the polynomial kernel. This kernel is still based on inner products, but the square introduces nonlinear factors, so its spectrum does not have the same clean cut-off as the linear kernel. Figure 2.1(c) shows the RBF kernel. Here the decay depends more on the geometry of the data and the value of γ, so this is the most interesting case out of all three kernels. Although not shown, repeating this calculation for larger subsamples gave similar looking decay patterns. This concerns only the relative spectrum $\frac{\sigma_i(K)}{\sigma_1(K)}$ so the unnormalised singular values may still increase in absolute scale as n increases.

The decay in these plots motivates replacing K by a lower-rank matrix $\widetilde{K}$ in later chapters. The goal is to keep the main spectral information needed for the KRR computation while reducing the cost.

2.5 Runtime and accuracy experiments

We now check the computational cost in this section. Constructing a full kernel matrix means evaluating pairwise similarities between observations so the expected cost is quadratic in n. Solving the full KRR has cubic asymptotic cost if we use Cholesky or some other direct solver (Bach, 2013; Alaoui and Mahoney, 2015). However in the tested range the construction of K can still take longer than the solve. This does not contradict the asymptotic costs. It just means the $O(n^2)$ construction curve can start much higher since every kernel entry has to be evaluated and stored while the solve is done by highly optimized linear algebra methods.

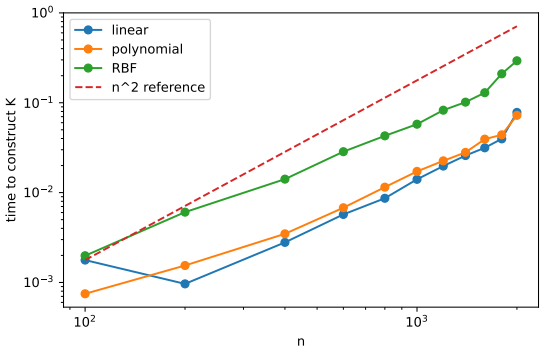


Figure 2.2: Time to construct the full kernel matrix for different values of n.

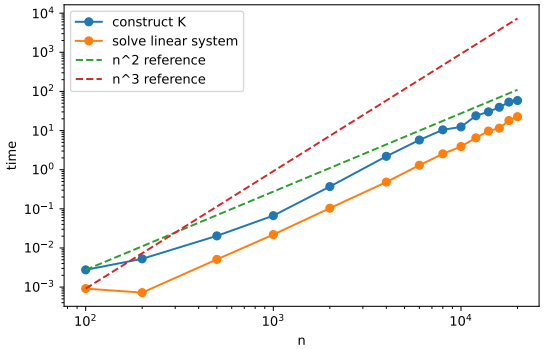


Figure 2.3: Time to construct K vs time to solve the full KRR system.

Figure 2.2 shows the construction time for the linear, polynomial and RBF kernels. The general behaviour follows the expected quadratic growth, but the constants are not the same. RBF kernel is slower since it is a more complex function than the other two.

Figure 2.3 compares the time to construct the RBF kernel matrix with the time to solve $(K + \lambda I)\alpha = y$. In the code, we measure the solve time by forming $K + \lambda I$, then applying a Cholesky factorization and two triangular solves. The construction time is larger for the increasing values of n which agrees with the theoretical expectation we have discussed above. This is one of the main reasons why column sampling is important: if only l columns are sampled, we need not form all of K which we will see in Chapter 3.

We now look at the behaviour of the full KRR solution as the regularization parameter λ changes. This is important because λ appears directly in the system $(K + \lambda I)\alpha = y$, so changing λ changes both the model and the conditioning of the linear system. Later, when K is replaced by an approximation $\tilde{K}$ in Nyström, the same parameter still appears in the approximation system. Therefore it is useful to first understand the effect of λ in the full KRR system.

For this experiment, we use a fixed train/test split of 800 training points and 200 test points, and measure the test error using MSE. The mean squared error (MSE) is defined as

$$\text{MSE} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (\hat{y}_i - y_i)^2 ,$$

where y_i is the true output and $\hat{y}_i$ is the predicted output for the ith test observation. Smaller values indicate more accurate test predictions. We used MSE since it is a natural metric for the regression error and it gives a simple baseline that later can be compared with the Nyström approximations.

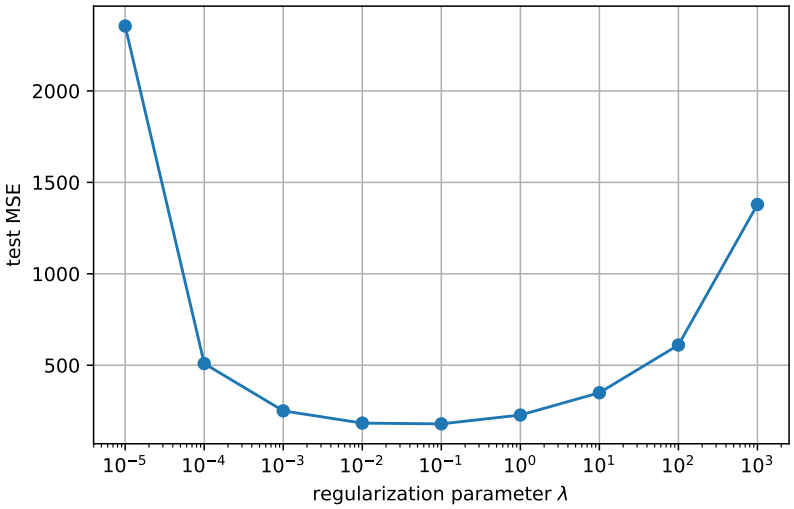


Figure 2.4: Test MSE for full KRR as λ changes.

Figure 2.4 shows that the test MSE depends strongly on λ. For very small values, especially $\lambda = 10^{-5}$ and $\lambda = 10^{-4}$, the test error is high. In this range the regularization term is too weak, so the solution can fit the training data too closely and generalize poorly. As λ increases, the test MSE drops sharply and reaches its lowest value around $\lambda = 10^{-1}$. After this point the error rises again, and for very large values such as $\lambda = 10^{2}$ and $\lambda = 10^{3}$, the penalty term is too strong and the fitted function becomes too restricted. Thus the plot shows the trade-off controlled by λ: both very small or very large λ give less stable fits, small lambda underregularizes which causes overfitting and large lambda overregularizes which causes underfitting. The best value in this small experiment is in the middle of the tested range at roughly $\lambda = 0.01$ to 0.1.

The same experiment was repeated for several training sizes in Figure 2.5. We are interested in how the optimal regularization parameter changes with the training size.

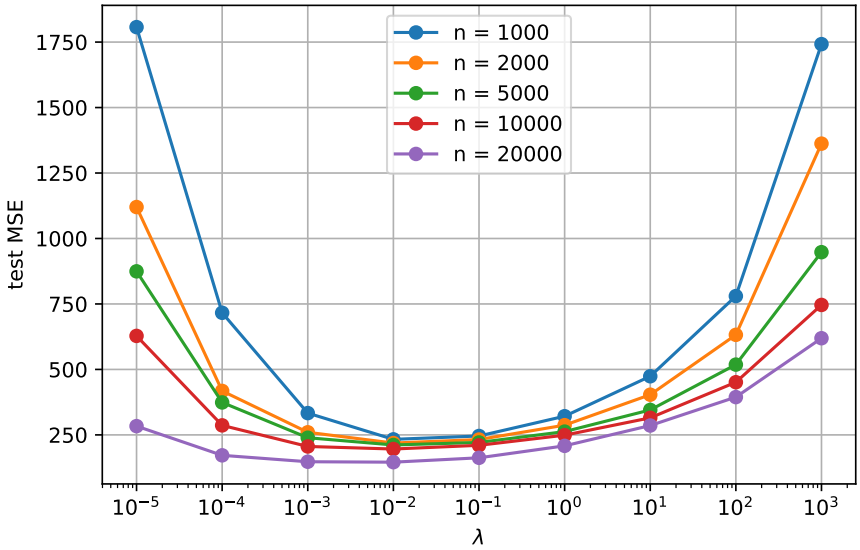


Figure 2.5: Test MSE against the KRR regularization parameter for different training sizes.

Figure 2.5 shows that the same U-shaped regularization behaviour remains the same as training size changes. As n becomes larger, the bottom of the curve becomes flatter and the optimal λ seems to become smaller. This indicates that the prediction error is less sensitive to small changes in λ near the best region as training data size becomes larger.

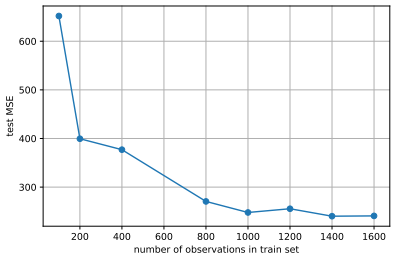


Figure 2.6: Test MSE as the number of training observations increases.

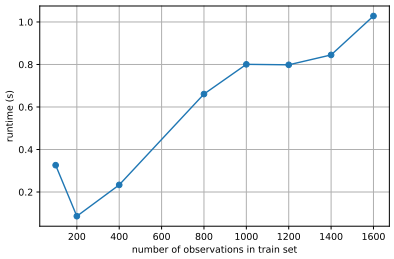


Figure 2.7: Runtime as the number of training observations increases.

Finally, we fix the test set and use increasingly larger subsets of the training set. This checks both the prediction error and the runtime as the number of training observations grows.

Figures 2.6 and 2.7 show the basic trade off. More training observations can improve the test error but the runtime increases at the same time. This is the practical situation where Nyström becomes very useful: ideally we want to keep majority of the predictive information while avoiding the computational cost of dealing with the full kernel matrix.

This closes the motivation for Chapter 3. We have shown how KRR becomes a finite-dimensional kernel matrix problem, why the full matrix is expensive and why low-rank structure is worth exploiting. Chapter 3 therefore introduces the Nyström method as a way to approximate the positive semidefinite kernel matrix K using only selected columns.

3 Nyström Approximation

The approach originates from a numerical procedure introduced by Nyström for solving integral equations (Nyström, 1930). In modern kernel approximation, it constructs a low rank approximation of a large positive semidefinite matrix by using only a smaller part of it (Williams and Seeger, 2000; Drineas et al., 2005). In this project the positive semidefinite matrix is the kernel matrix K, but it is useful to first describe the construction for a general positive semidefinite matrix A. The aim is to build a low rank approximation $\tilde{A}$ without using all of A in a full direct factorization.

3.1 General Nyström form

We now look at the general Nyström form. Let $A \in \mathbb{R}^{n\times n}$ and denote $\succeq$ as the sign for positive semidefiniteness where $A \succeq 0$ means A is positive semidefinite. And let $X \in \mathbb{R}^{n\times l}$ be a test matrix. The columns of X specify the directions through which the matrix A is probed and the product AX is the resulting sample matrix. The column space of AX contains enough information to construct an accurate low rank approximation if these sampled directions capture the dominant subspace of A.

Following the general formulation in randomized numerical linear algebra, we let $A \in \mathbb{R}^{n\times n}$ where $A \succeq 0$, and let $X \in \mathbb{R}^{n\times l}$ be a test matrix, the Nyström approximation of A with respect to X is

$$A\langle X\rangle = (AX)(X^TAX)^\dagger(AX)^T. \tag{3.1}$$

Here $(X^TAX)^\dagger$ denotes the Moore–Penrose pseudoinverse. The pseudoinverse generalizes the inverse to matrices that may be singular and it agrees with the usual inverse when X^TAX is invertible (Penrose, 1955). The test matrix X is the thing we use to probe or sketch A by forming AX (Martinsson and Tropp, 2020). Later on when we sample columns this matrix becomes a sampling matrix that selects columns of the kernel matrix. The matrix AX contains the information we collect from A. The smaller matrix

$$X^TAX \in \mathbb{R}^{l\times l}$$

describes how the sampled directions interact with each other.

Equation (3.1) shows why the approximation has rank at most l since it is a product involving the $n \times l$ matrix AX, so

$$\operatorname{rank}(A\langle X\rangle) \le l.$$

Therefore, when $l \ll n$, the approximation is much cheaper to store and compute with than the full matrix A.

There is also a useful geometric representation of this formula (Martinsson and Tropp, 2020, Section 14). Since A is positive semidefinite, it has a positive semidefinite square root $A^{1/2}$ and $A = A^{1/2}A^{1/2}$. Therefore

$$AX = A^{1/2}(A^{1/2}X)$$

and

$$(AX)^T = (A^{1/2}X)^TA^{1/2}.$$

Also,

$$X^TAX = X^TA^{1/2}A^{1/2}X = (A^{1/2}X)^T(A^{1/2}X).$$

Substituting these into the Nyström approximation formula gives

$$A\langle X\rangle = A^{1/2}[(A^{1/2}X)((A^{1/2}X)^T(A^{1/2}X))^\dagger(A^{1/2}X)^T]A^{1/2}.$$

If we set

$$Y = A^{1/2}X.$$

Then

$$Y^TY = (A^{1/2}X)^T(A^{1/2}X) = X^TAX.$$

Then the expression inside the bracket becomes

$$Y(Y^TY)^\dagger Y^T.$$

We see that this is the standard formula for the orthogonal projector onto the column space of Y (Trefethen and Bau, 1997). Therefore, if we let P_Y denote this projector, then

$$A\langle X\rangle = A^{1/2}P_YA^{1/2}.$$

Since $Y = A^{1/2}X$, we can also just write this as $P_{A^{1/2}X}$. Hence

$$A\langle X\rangle = A^{1/2}P_{A^{1/2}X}A^{1/2}.$$

This projector representation can now be used to derive an expression for the Nyström approximation error $A - A\langle X\rangle$. Since $A = A^{1/2}IA^{1/2}$, we have

$$A - A\langle X\rangle = A^{1/2}IA^{1/2} - A^{1/2}P_{A^{1/2}X}A^{1/2}.$$

Factoring out $A^{1/2}$ on both sides gives

$$A - A\langle X\rangle = A^{1/2}(I - P_{A^{1/2}X})A^{1/2}. \tag{3.2}$$

So the error is the part of A that is not captured by the subspace spanned by the columns of $A^{1/2}X$. Since $P_{A^{1/2}X}$ is an orthogonal projector and therefore positive semidefinite, this representation also shows that $A\langle X\rangle \succeq 0$.

Equation (3.2) is the Schur complement viewpoint used by Martinsson and Tropp (Martinsson and Tropp, 2020). This means that the Nyström approximation keeps the part of A which is seen through the subspace generated by X. We now give a formal theorem of this.

Theorem 3.1 (Nyström error representation; Martinsson and Tropp, 2020)**.** *Let $A \in \mathbb{R}^{n\times n}$ be positive semidefinite and let $X \in \mathbb{R}^{n\times l}$. Then*

$$A - A\langle X\rangle = A^{1/2}(I - P_{A^{1/2}X})A^{1/2}.$$

In particular,

$$A - A\langle X\rangle \succeq 0,$$

so

$$0 \preceq A\langle X\rangle \preceq A.$$

We have proved parts of this theorem above, to complete it we see that since $I - P_{A^{1/2}X}$ is an orthogonal projector, it is positive semidefinite. Thus $A - A\langle X\rangle$ is positive semidefinite, i.e. $A - A\langle X\rangle \succeq 0$. Rearranging that gives the last statement of the theorem where the notation $A \preceq B$ means that $B - A$ is positive semidefinite.

Theorem 3.1 identifies the error matrix of the Nyström approximation. The left hand side $A - A\langle X\rangle$ is exactly the part of A that is not captured by the approximation. The right hand side shows that this error is determined by the complementary projector $I - P_{A^{1/2}X}$. Since $P_{A^{1/2}X}$ projects onto the column space of $A^{1/2}X$, the matrix $I - P_{A^{1/2}X}$ projects onto complementary part of that space in the whole space. Therefore the theorem says that the

Nyström error comes from the directions of A that are missed by the subspace generated by $A^{1/2}X$ (Martinsson and Tropp, 2020).

This shows why the choice of X matters. If the columns of $A^{1/2}X$ capture the important directions of A, then the missed part $A^{1/2}(I - P_{A^{1/2}X})A^{1/2}$ should be smaller. If they miss important directions, then the approximation error can be large. In the column-sampling case, this means that the quality of the Nyström approximation depends on which columns are sampled. This motivates later sections to compare uniform sampling, leverage-score sampling and approximate leverage-score sampling.

The statement $0 \preceq A\langle X\rangle \preceq A$ says that the Nyström approximation does not create a matrix larger than A in the positive semidefinite order. This means that it does not invent or exaggerate in any directions in A and it approximates A from below. In the KRR setting this matters because replacing K by $\tilde{K}$ changes the regularized system from

$$(K + \lambda I)\alpha = y$$

to

$$(\tilde{K} + \lambda I)\tilde{\alpha} = y.$$

Since $\tilde{K} \preceq K$, the approximation changes both the estimator and the conditioning of the linear system. In directions where $\tilde{K}$ approximates K well, the solution should be close. In directions where $\tilde{K}$ loses a lot of information, the λI regularization term dominates more. This is one reason why the number of sampled columns l, the sampling strategy and the regularization parameter λ have to be studied together.

3.2 Randomized Nyström approximation

We now discuss more on the test matrix X mentioned above that is used to probe the important directions of A. The randomized Nyström approximation draws a random matrix $X \in \mathbb{R}^{n\times l}$ and forms $Y = AX$. Then the Nyström approximation is

$$\hat{A} = Y(X^TY)^\dagger Y^T.$$

This is the randomized Nyström approximation described by Martinsson and Tropp (Martinsson and Tropp, 2020). The idea of a random test matrix was first introduced through the randomized rangefinder. In theory we can draw a random matrix X and form a sample matrix such as $Y = BX$ for some matrix B, whose columns are used to capture an important subspace of the target matrix (Halko et al., 2011; Martinsson and Tropp, 2020). A Gaussian test matrix is a standard theoretical choice where its entries are independent Gaussian random variables, so BX gives random linear combinations of the columns of B. This explains why randomization can be used to find a useful low-dimensional subspace. Nyström approximation therefore uses a similar idea but in this case the target matrix is positive semidefinite (Martinsson and Tropp, 2020, Section 14).

However, for kernel matrices, a dense Gaussian test matrix is usually not practical. Kernel matrices are expensive because even forming their entries requires $\mathcal{O}(n^2)$ kernel evaluations. In the coming sections we will see that the Nyström approximation is usually implemented by sampling columns of the kernel matrix. This is equivalent to using a column-selection matrix as the test matrix. Let $I = \{i_1, \dots, i_l\} \subset \{1, \dots, n\}$ be the selected column indices and let $S = [e_{i_1} \ \cdots \ e_{i_l}] \in \mathbb{R}^{n\times l}$. Define the sampling matrix

$$Y = KS = K_{:,I}.$$

Therefore the Gaussian test matrix explains the general randomized sketching idea, while the column-selection matrix gives the practical Nyström approximation used in the experiments (Williams and Seeger, 2000; Kumar et al., 2012; Gittens and Mahoney, 2016).

3.3 Nyström approximation and the KRR system

We now discuss the practical implementation of the Nyström approximation. Since K is symmetric positive semidefinite, it has an eigendecomposition

$$K = U\Lambda U^T,$$

where U is an orthogonal matrix and all eigenvalues in Λ are real. If we sort the eigenvalues from largest to smallest and let $U_1 \in \mathbb{R}^{n\times l}$ contain the eigenvectors corresponding to the l largest eigenvalues. The associated eigenvalues form $\Lambda_1 \in \mathbb{R}^{l\times l}$. We then partition

$$U = \begin{bmatrix} U_1 & U_2 \end{bmatrix}, \qquad \Lambda = \begin{bmatrix} \Lambda_1 & 0 \\ 0 & \Lambda_2 \end{bmatrix}.$$

Therefore

$$K = U_1\Lambda_1 U_1^T + U_2\Lambda_2 U_2^T.$$

If the eigenvalues in Λ_2 are small, then the second term contributes less to the matrix. And since $\Lambda_1 \in \mathbb{R}^{l\times l}$, the ideal rank-$l$ approximation is $K \approx U_1\Lambda_1 U_1^T$. This motivates the approximation idea: instead of using all directions of K, we would like to keep only the important directions. We have seen this idea in the section on the general Nyström form, where the approximation error is determined by the directions that are missed by the subspace generated by X. In this case, the important directions are those corresponding to the eigenvectors in U_1 (Eckart and Young, 1936).

However, computing U_1 and Λ_1 directly would require spectral information from the full kernel matrix. This is not practical if the goal is to avoid computing with all of K. Thus we construct a low rank approximation from sampled columns with $\tilde{K} = CW^\dagger C^T$. Here $C = K_{:,I}$ contains the sampled columns, and $W = K_{I,I}$ is the smaller intersection matrix. This is not the exact truncated eigendecomposition, but it is a column sampling way to approximate the important low rank part of K as we will see in the next section (Williams and Seeger, 2000; Drineas et al., 2005; Kumar et al., 2012; Gittens and Mahoney, 2016; Musco and Musco, 2017).

3.4 Column sampling

We now look at column sampling. Let

$$I = \{i_1, \ldots, i_l\} \subset \{1, \ldots, n\}$$

be the selected column indices, this set of columns are also called *landmarks*. Define the sampling matrix

$$S = \begin{bmatrix} e_{i_1} & e_{i_2} & \cdots & e_{i_l} \end{bmatrix} \in \mathbb{R}^{n\times l},$$

where e_i is the i-th standard basis vector. Taking $X = S$ in the general Nyström formula gives

$$\tilde{K} = K\langle S\rangle = (KS)(S^TKS)^\dagger(KS)^T.$$

Now define

$$C = KS = K_{:,I} \in \mathbb{R}^{n\times l}$$

and

$$W = S^TKS = K_{I,I} \in \mathbb{R}^{l\times l}.$$

Then the Nyström approximation becomes

$$\tilde{K} = CW^\dagger C^T.$$

Equivalently,

$$\tilde{K} = K_{:,I}(K_{I,I})^{\dagger}K_{I,:}.$$

This is the form used in this project. Instead of forming or storing all of K, the approximation is built from the sampled columns $K_{:,I}$ and the smaller intersection matrix $K_{I,I}$. The notation $K_{:,I}$ means that we keep all rows of K, but only the columns whose indices are in I. In other words,

$$C = K_{:,I} = \begin{bmatrix} K_{1,i_1} & K_{1,i_2} & \cdots & K_{1,i_l} \\ K_{2,i_1} & K_{2,i_2} & \cdots & K_{2,i_l} \\ \vdots & \vdots & & \vdots \\ K_{n,i_1} & K_{n,i_2} & \cdots & K_{n,i_l} \end{bmatrix}$$

Similarly, $K_{I,I}$ means that both the rows and columns only contain that of the selected index set I:

$$W = K_{I,I} = \begin{bmatrix} K_{i_1,i_1} & K_{i_1,i_2} & \cdots & K_{i_1,i_l} \\ K_{i_2,i_1} & K_{i_2,i_2} & \cdots & K_{i_2,i_l} \\ \vdots & \vdots & & \vdots \\ K_{i_l,i_1} & K_{i_l,i_2} & \cdots & K_{i_l,i_l} \end{bmatrix}$$

If W is invertible then the formula simplifies to

$$\tilde{K} = CW^{-1}C^T.$$

However in practice $W^{\dagger}$ is safer to write since W can be singular or nearly singular.

The column-sampling formula above also gives a simple Nyström algorithm. The steps are:

1. choose l landmark indices $I = \{i_1, \ldots, i_l\}$;
2. form matrix $C = K_{:,I}$ and $W = K_{I,I}$;
3. compute the pseudoinverse $W^{\dagger}$;
4. approximate K by $\tilde{K} = CW^{\dagger}C^T$.

The approximate KRR system obtained from the Nyström approximation is

$$(CW^{\dagger}C^T + \lambda I)\tilde{\alpha} = y.$$

If we first form the full matrix $CW^{\dagger}C^T$, then we again have an $n \times n$ matrix. This is useful in small experiments, but it does not use the full advantage of the Nyström representation. What we can do is to keep C and W, then reduce the solve to a smaller system.

First we introduce a small shift $\nu > 0$. This shift is used for numerical stability (Martinsson and Tropp, 2020, Algorithm 16). Define the shifted column sampling matrix

$$C_{\nu} = C + \nu S.$$

Since $C = KS$, this can also be written as

$$C_{\nu} = KS + \nu S = (K + \nu I)S.$$

Thus the shift corresponds to applying the column sampling to $K + \nu I$ instead of directly to K.

The corresponding shifted intersection matrix is

$$S^T C_{\nu} = S^T(C + \nu S) = S^T K S + \nu S^T S.$$

Since S is a column selecting matrix, its columns are orthonormal coordinate vectors. So $S^T S = I_l$ and therefore

$$S^T C_\nu = W + \nu I_l = W_\nu.$$

The shift makes W_ν positive definite in exact arithmetic, so we can compute its Cholesky factorization

$$W_\nu = LL^T,$$

where $L \in \mathbb{R}^{l \times l}$ is a lower triangular matrix.

Next, let

$$B = C_\nu L^{-T}.$$

We have

$$BB^T = C_\nu L^{-T} L^{-1} C_\nu^T = C_\nu (LL^T)^{-1} C_\nu^T = C_\nu W_\nu^{-1} C_\nu^T.$$

Thus BB^T is the shifted Nyström approximation, but stored through the thin factor $B \in \mathbb{R}^{n \times l}$ rather than as a full $n \times n$ matrix.

Now we can compute a singular value decomposition of B of the form $B = U\Sigma V^T$, where

$$U \in \mathbb{R}^{n \times l}, \qquad \Sigma \in \mathbb{R}^{l \times l}, \qquad V \in \mathbb{R}^{l \times l}.$$

Then

$$BB^T = U\Sigma V^T V \Sigma U^T = U\Sigma^2 U^T.$$

Therefore the singular values of B give the eigenvalues of the shifted low rank approximation.

The shift ν was introduced only for the Cholesky factorization, so it is removed from the eigenvalues after the SVD. If

$$\Sigma^2 = \operatorname{diag}(\sigma_1^2, \ldots, \sigma_l^2),$$

Let

$$\eta_j = \max(0, \sigma_j^2 - \nu),$$

and

$$\Lambda = \operatorname{diag}(\eta_1, \ldots, \eta_l).$$

With a slight abuse of notation, we reuse U and Λ for the factors of the Nyström approximation instead of the eigendecomposition of K from Section 3.3. The final factorized Nyström approximation is then

$$\tilde{K} = U\Lambda U^T.$$

This is the form used for the KRR solve. The large matrix $\tilde{K}$ is not explicitly formed, it is stored by the matrix U with orthonormal columns and the diagonal matrix Λ.

Using the factorized representation in $(\tilde{K} + \lambda I)\tilde{\alpha} = y$, we have

$$(U\Lambda U^T + \lambda I)\tilde{\alpha} = y.$$

We now solve this system using the orthogonality of U. Complete the columns of U to an orthonormal basis:

$$Q = \begin{bmatrix} U & U_\perp \end{bmatrix},$$

where

$$U^T U = I, \qquad U_\perp^T U_\perp = I, \qquad U^T U_\perp = 0.$$

Therefore

$$UU^T + U_\perp U_\perp^T = I.$$

Since $\tilde{K} = U\Lambda U^T$, we can write

$$\tilde{K} = \begin{bmatrix} U & U_\perp \end{bmatrix} \begin{bmatrix} \Lambda & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} U^T \\ U_\perp^T \end{bmatrix}.$$

Adding the regularization term gives

$$\tilde{K} + \lambda I = \begin{bmatrix} U & U_\perp \end{bmatrix} \begin{bmatrix} \Lambda + \lambda I & 0 \\ 0 & \lambda I \end{bmatrix} \begin{bmatrix} U^T \\ U_\perp^T \end{bmatrix}.$$

Then we have

$$(\tilde{K} + \lambda I)^{-1} = \begin{bmatrix} U & U_\perp \end{bmatrix} \begin{bmatrix} (\Lambda + \lambda I)^{-1} & 0 \\ 0 & \frac{1}{\lambda} I \end{bmatrix} \begin{bmatrix} U^T \\ U_\perp^T \end{bmatrix}.$$

Multiplying this by y gives

$$\tilde{\alpha} = U(\Lambda + \lambda I)^{-1} U^T y + \frac{1}{\lambda} U_\perp U_\perp^T y.$$

Using

$$U_\perp U_\perp^T = I - UU^T,$$

we get

$$\tilde{\alpha} = U(\Lambda + \lambda I)^{-1} U^T y + \frac{1}{\lambda}(I - UU^T) y.$$

In implementation, neither $U_\perp$ nor $I - UU^T$ needs to be formed. Since

$$\Lambda = \operatorname{diag}(\eta_1, \ldots, \eta_l),$$

the solution is equivalently to

$$\tilde{\alpha} = \frac{1}{\lambda} y - U \operatorname{diag}\left(\frac{\eta_j}{\lambda(\eta_j + \lambda)} \right) U^T y.$$

The computational cost after the columns have been selected is much smaller than a dense direct solve. The Cholesky factorization of $W_\nu \in \mathbb{R}^{l \times l}$ costs $O(l^3)$, forming $B = C_\nu L^{-T}$ costs $O(nl^2)$, and the thin SVD of $B \in \mathbb{R}^{n \times l}$ costs about $O(nl^2)$ (Trefethen and Bau, 1997). After having U and Λ, evaluating $\tilde{\alpha}$ costs $O(nl)$. Hence the dominant cost is

$$O(nl^2 + l^3),$$

which is much smaller than solving the full dense KRR system when $l \ll n$.

3.4.1 Uniform sampling

The baseline method is uniform sampling. This means choosing l column indices from $\{1, \ldots, n\}$ uniformly at random. It is easy to implement and cheap because no importance scores have to be computed. Its weakness is that it treats all columns equally. If some columns are much more informative than others, uniform sampling may need a larger l to reach the same accuracy as a smarter strategy (Kumar et al., 2012; Gittens and Mahoney, 2016).

Before stating the theorem for the uniform sampling error bound, we define the leverage scores and coherence used in the bound.

Let $K \in \mathbb{R}^{n \times n}$ be a positive semidefinite kernel matrix with eigendecomposition

$$K = U\Lambda U^T.$$

Let $U_k \in \mathbb{R}^{n\times k}$ contain the eigenvectors corresponding to the k largest eigenvalues of K. Following Gittens and Mahoney, the *statistical leverage score* of row or column i relative to the best rank-k approximation is defined as

$$s_i = \|U_k(i,:)\|_2^2.$$

These scores measure how much the i-th coordinate contributes to the leading k-dimensional eigenspace. Since U_k has orthonormal columns, the scores satisfy

$$\sum_{i=1}^{n} s_i = k.$$

The *coherence* is defined as

$$\mu = \frac{n}{k} \max_{1\le i\le n} s_i.$$

Thus μ is the scaled version of the largest leverage score (Drineas et al., 2012; Gittens and Mahoney, 2016).

From the same eigendecomposition, the best rank-k approximation is

$$K_k = U_k \Lambda_k U_k^T,$$

where U_k contains the eigenvectors corresponding to the k largest eigenvalues as seen before. This matrix is the best rank-k approximation to K in spectral norm. In the Nyström settings, k is therefore the optimal rank we compare against while l is the number of columns actually sampled. In practice l should be larger than k, since random column sampling needs to oversample to capture the important k-dimensional subspace with high probability.

The leverage score measures how concentrated the leading eigenspace is in a few rows. If the scores are all similar, then μ is not large and uniform sampling has a better chance of working. If a few scores are much larger than the rest, then uniform sampling can easily miss important columns. Thus uniform sampling can be reasonable in some datasets where the leverage scores are fairly uniform. In other datasets where the leverage scores are highly nonuniform, uniform sampling can be much worse (Kumar et al., 2012; Gittens and Mahoney, 2016). This is why uniform sampling is a natural baseline method but not necessarily the best method.

Finally we have the theorem, and it uses two probability parameters. The failure probability δ controls how likely it is that the random sampling guarantee fails, so a smaller δ means higher confidence. The accuracy factor ϵ controls the sampling condition, a smaller ϵ represents a stronger guarantee and therefore requires more columns to be sampled.

Theorem 3.2 (Uniform sampling; Gittens and Mahoney, 2016, Lemma 8)**.** *Let $K \in \mathbb{R}^{n\times n}$ be positive semidefinite, and let K_k be its best rank-k approximation. Fix a failure probability δ and an accuracy factor ϵ both in $(0,1)$. Suppose l columns are sampled uniformly at random, then*

$$C = KS = K_{:,I}, \qquad W = S^T KS = K_{I,I}.$$

If

$$l \ge 2\mu\epsilon^{-2} k \ln(k/\delta),$$

then, with probability at least $1-3\delta$, the Nyström approximation $\tilde{K} = CW^{\dagger}C^T$ satisfies the spectral norm bound

$$\|K - \tilde{K}\|_2 \le \left(1 + \frac{n}{(1-\epsilon)l}\right) \|K - K_k\|_2.$$

This bound for l fits the intuition as it shows us when μ is large we need to sample more columns. This is exactly the point that uniform sampling does not know which columns are important so when the coherence is large it needs more columns to capture the essential structure.

3.4.2 Leverage score sampling

Uniform sampling is cheap but it ignores the structure of K. Leverage score sampling is the opposite extreme, it tries to sample columns according to how important they are in the leading eigenspace.

The full leverage score version is the most informed method. It uses all of K first:

1. construct the full kernel matrix K;
2. compute the leading eigenvectors U_k where U_k comes from the eigendecomposition $K = U\Lambda U^T$ as before;
3. compute the squared row norms $s_i = \|U_k(i,:)\|_2^2$;
4. since U_k has k orthonormal columns, $\sum_{j=1}^{n} s_j = \|U_k\|_F^2 = k$, compute exact leverage score probabilities $p_i = \frac{s_i}{\sum_j s_j} = \frac{1}{k}\|U_k(i,:)\|_2^2$;
5. sample columns with probabilities p_i.

(Drineas et al., 2012; Gittens and Mahoney, 2016)

This method is expensive because it requires the full matrix and the leading eigenvectors. But it is still useful as a reference method. It says what the informed sampling distribution would be if the important eigenspace was already known.

Theorem 3.3 (Leverage score sampling; Gittens and Mahoney, 2016, Lemma 5)**.** *Let $K \in \mathbb{R}^{n\times n}$ be positive semidefinite and let U_k contain the leading k eigenvectors. Suppose columns are sampled with probabilities satisfying*

$$p_i \geq \frac{\beta}{k}\|U_k(i,:)\|_2^2$$

for some $\beta \in (0,1]$. Fix a failure probability $\delta \in (0,1]$ and an approximation factor $\epsilon \in (0,1]$. If

$$l \geq \frac{3200}{\beta\epsilon^2} k \log\left(\frac{4k}{\beta\delta}\right),$$

then with probability at least $1 - 6\delta - 0.6$, the Nyström approximation satisfies

$$\|K - \tilde{K}\|_2 \leq \|K - K_k\|_2 + \epsilon^2\|K - K_k\|_*,$$

where $\|\cdot\|_*$ denotes the nuclear norm which is the sum of a matrix's singular values.

If we denote the exact leverage score probability as

$$q_i = \frac{1}{k}\|U_k(i,:)\|_2^2$$

and look at the probability bound. The condition $p_i \geq \beta q_i$ and $\beta \in (0,1]$ allows the theorem to use approximate probabilities, i.e. approximate leverage scores which will be seen later. The parameter β measures their quality where $\beta = 1$ gives the exact leverage score distribution, while a smaller β allows greater underestimation and thus requires more sampled columns (Gittens and Mahoney, 2016).

This theorem can be compared with the uniform sampling theorem. In the uniform sampling theorem, the required number of columns depends on the coherence μ, because uniform sampling does not know which columns are important. In the leverage score result, the sampling probabilities depend on the row norm scores $\|U_k(i,:)\|_2^2$, so the sample-size condition no longer has the same coherence factor (Gittens and Mahoney, 2016).

The term $\|K - K_k\|_2$ is the spectral norm error of the best rank-k approximation, so this is the error that's unavoidable if only rank k information is kept. The theorem says that leverage score sampling can get close to this best rank-k error, with an additional tail term $\epsilon^2\|K - K_k\|_*$. This extra term depends on the nuclear norm of the remaining tail. Therefore, if the eigenvalues after the first k are small, the additive term is also small.

The theoretical advantage of leverage score sampling is therefore that it adapts to the nonuniformity of the leading eigenspace. Unlike the uniform sampling condition in Theorem 3.2, the required sample size does not contain the coherence factor μ. Therefore when the leading eigenspace is concentrated in a small number of coordinates, leverage score sampling can target these important columns directly.

4 Informed practical column sampling

The previous chapter introduced the Nyström approximation and showed why the choice of sampled columns matters. Uniform sampling is cheap but it does not use any information about which columns of the matrix are important. Exact leverage score sampling determines the importance of columns but it is too expensive as it requires the leading eigenvectors of the full kernel matrix which is the thing we are explicitly trying to avoid forming (Drineas et al., 2012; Alaoui and Mahoney, 2015). This chapter studies the middle ground: sampling methods that try to use importance information without paying the full cost of extracting spectral information.

The chapter has two sections. First, it introduces approximate leverage score sampling, where a small initial uniform sample is used to estimate column importance. Second, it introduces recursive ridge leverage score sampling, based on Algorithm 2 of Musco and Musco (2017). Chapter 5 compares these three sampling methods inside the KRR problem, in terms of approximation quality and prediction accuracy against runtime.

4.1 Approximate leverage score sampling

The heuristic used in this project is motivated by approximate leverage score estimation (Drineas et al., 2012; Kumar et al., 2012; Gittens and Mahoney, 2016). A simple version of approximate leverage score sampling is the following: First, sample a small number l_0 of columns uniformly. Second, use these columns to estimate which rows or columns of the kernel matrix are important. Finally, sample the remaining $l - l_0$ columns according to these estimated scores. For example, if $l = 100$ and $l_0 = 5$, this means sampling 5 columns uniformly and then sampling 95 more columns according to approximate leverage scores.

Let

$$I_0 = \{i_1, \ldots, i_{l_0}\}$$

be the initial uniformly sampled index set and define

$$K_s = K_{:,I_0} \in \mathbb{R}^{n \times l_0}.$$

We compute a thin singular value decomposition

$$K_s = U_s \Sigma_s V_s^T.$$

The columns of U_s form an orthonormal basis for the column space found from the initially sampled columns. This small subspace is used as a rough approximation to the important part of the column space of the full matrix K. The approximate leverage score of row or column i is computed as

$$\hat{s}_i = \|U_s(i,:)\|_2^2$$

(Drineas et al., 2012; Gittens and Mahoney, 2016). Similar to exact leverage scores, the approximate scores are then normalized to form a probability distribution

$$p_i = \frac{\hat{s}_i}{\sum_{j=1}^{n} \hat{s}_j},$$

and $l - l_0$ further columns are sampled according to these probabilities. The final index set is thus $I = I_0 \cup I_1$, where I_1 contains the extra sampled column indices. The Nyström approximation is therefore

$$\tilde{K} = K_{:,I}(K_{I,I})^{\dagger} K_{I,:}.$$

We now discuss the diagnostics for approximate leverage score sampling to see whether the estimated scores are reliable by comparing them to the exact scores and also check how initial sampling affects the approximation.

4.1.1 Comparison to exact leverage scores

With this experiment, we aim to explain why approximate leverage sampling may or may not improve over uniform sampling. We randomly select 300 observations from the standardized Superconductivity data and take the corresponding submatrix of the RBF kernel matrix

$$K_{\text{sub}} \in \mathbb{R}^{300\times 300}.$$

The size $n = 300$ is chosen to keep the exact eigendecomposition computationally cheap since we only use this experiment as a diagnostic comparison between exact and approximate leverage scores. Thus there are 300 exact scores and each score measures importance relative to the leading 50-dimensional eigenspace. For approximate leverage score sampling we use an initial uniform sample of $l_{\text{ini}} = 100$ columns. The approximate scores are then compared with the exact scores. For this comparison, only the first $r = 50$ columns of U_s are used when computing the approximate scores.

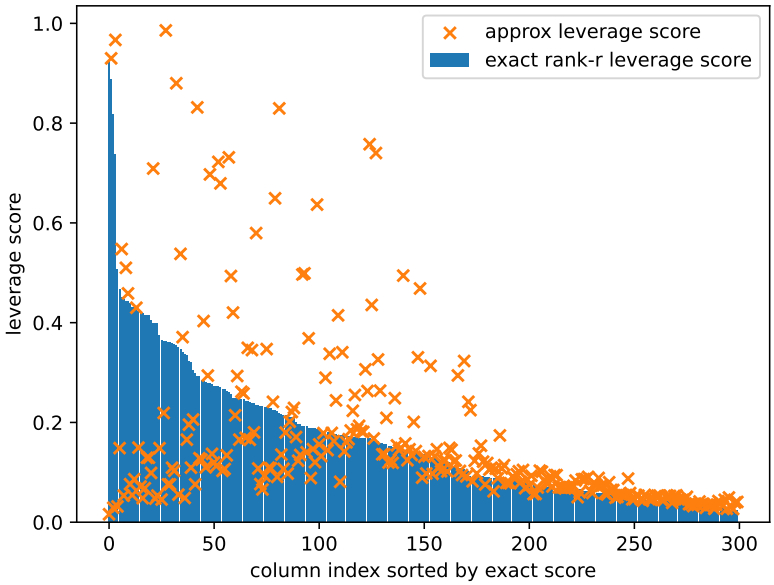


Figure 4.1: Exact rank-r leverage scores versus approximate leverage scores on a 300×300 kernel submatrix.

Figure 4.1 shows the exact scores as bars and the approximate scores as points, sorted by the exact score. If the approximation was perfect, the points would lie close to the tops of the bars. In this plot it seems that the approximate scores capture the broad scale but are very noisy. Some columns with high exact score are underestimated and some lower-score columns are overestimated. This shows why approximate leverage sampling is not guaranteed to beat uniform sampling in the final experiment as the score estimation step may contain a lot of error.

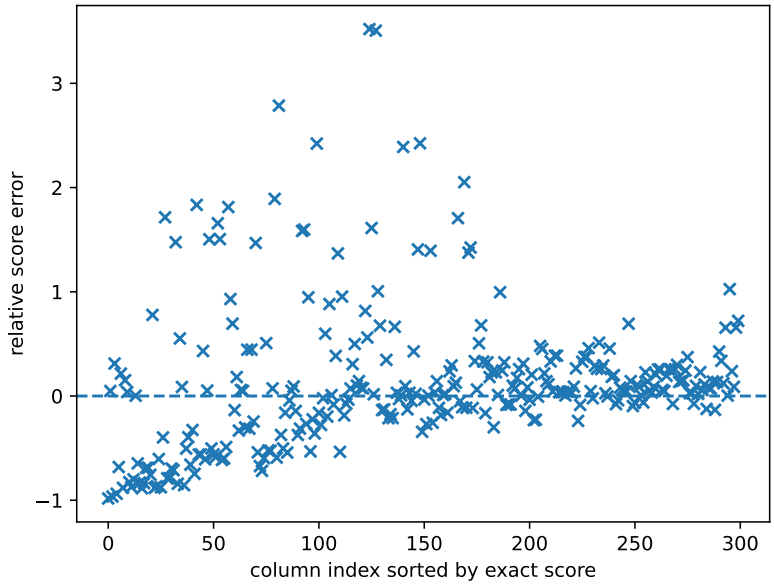


Figure 4.2: Relative error of approximate leverage scores compared with exact rank-r scores.

Figure 4.2 shows the relative score error and clearly the errors are not uniformly small. This confirms our interpretation earlier, it seems that the initial sketch does not recover the leading eigenspace perfectly. Since the sampling probabilities are based on these scores, inaccurate scores can reduce the advantage over uniform sampling.

Next we vary l_{ini}, with the target rank r is fixed. Increasing l_{ini} gives more information about the column space, but it also makes the SVD step more expensive.

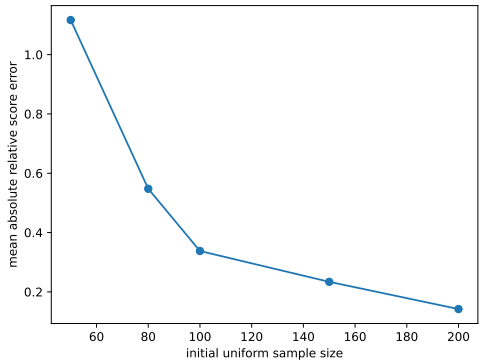


Figure 4.3: Mean relative score error against the initial sample size.

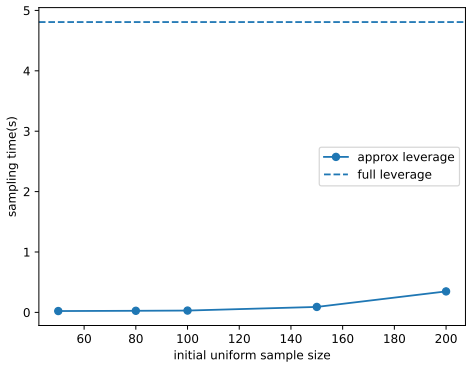


Figure 4.4: Score estimation time against the initial sample size.

Figures 4.3 and 4.4 show the expected trade off. As l_{ini} increases, the mean score error decreases. At the same time, the time required to estimate the scores increases. Next we investigate the effect of l_{ini} on the actual Nyström approximation while the final number of sampled columns is fixed.

4.1.2 Initial sample sizes and error

Finally, the effect of l_{ini} is tested on the actual Nyström approximation while the final number of sampled columns is fixed. We used a small subsample of size $n = 2000$ and fixed the final number of sampled columns at $l = 300$. The initial uniform sample size l_{ini} is varied from 50 to 200. The Nyström approximation error is measured as the relative Frobenius norm error. The experiment was repeated five times with independent random column selections, and the mean error was reported.

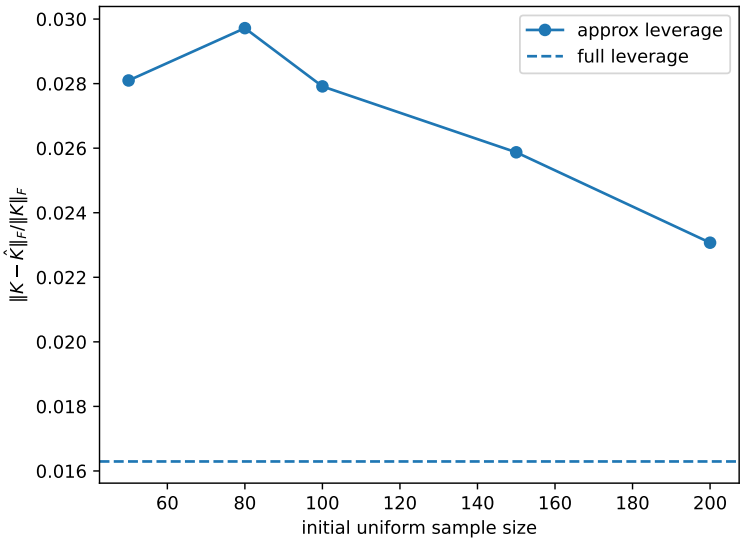


Figure 4.5: Mean Nyström approximation error against the initial uniform sample size over five repetitions, with fixed final sample size.

Figure 4.5 shows that improving score accuracy does not automatically give a monotone improvement in the mean Nyström error. The final sample size is fixed, so increasing l_{ini} also reduces the number of extra columns selected from the approximate scores. The result is roughly a trade off as a larger initial stage seems to give better score estimates, but it leaves less budget for the second stage. Repeated five-cycle experiment showed that the mean error still oscillates as l_{ini} increases, so the improvement remains nonmonotonic even after averaging over five repetitions.

4.1.3 Limitations of approximate leverage sampling

The approximate leverage method we used here is a crude practical compromise between uniform sampling and exact leverage score sampling which means it is not without limitations.

Indeed, the quality of these approximate scores depends entirely on whether the initial uniform sample captures the important directions of the full matrix. The method has the following dependency:

$$\text{good initial sketch} \Longrightarrow \text{useful approximate scores} \Longrightarrow \text{good final sampling.}$$

On the other hand,

$$\text{bad initial sketch} \Longrightarrow \text{misleading approximate scores} \Longrightarrow \text{possibly worse final sampling.}$$

This seems to create a circular difficulty. The purpose of leverage scores is to identify important columns, but the approximate leverage scores are themselves estimated from columns that were initially chosen uniformly. If an important direction is concentrated in a small number of columns and none of these columns is included in the initial sample, then this direction may not appear in the estimated subspace. The resulting approximate scores may therefore give low probability to exactly the columns that the method was intended to identify. Therefore the approximate leverage method may not be effective at all (Kumar et al., 2012; Gittens and Mahoney, 2016).

There are three main situations in which this method may operate. First, if the true leverage scores are already close to uniform, then uniform sampling is likely to work well. In this case, the approximate leverage method adds the cost of constructing the initial sketch and computing its singular value decomposition without gaining much over the uniform baseline.

Second, if the leverage scores are nonuniform and the initial uniform sample captures the dominant directions sufficiently well, then the approximate scores may successfully identify important indices. The additional columns may then be sampled more effectively than under a uniform distribution.

Third, if the leverage scores are strongly nonuniform but the initial sample misses important directions, then the approximate scores can be inaccurate. In this case, the second sampling stage may fail to improve over uniform sampling and may even produce a worse landmark set.

The theoretical advantage of leverage score sampling therefore does not automatically apply to every approximate leverage procedure. Exact leverage score sampling has guarantees because its probabilities are constructed from the true leading eigenspace. Theorem 3.3 also permits approximate leverage-score probabilities, provided that

$$p_i \geq \frac{\beta}{k}\|U_k(i,:)\|_2^2$$

for some known $\beta \in (0,1]$. Thus, similar guarantees can also hold for approximate scores, but only when their sampling probabilities satisfy this lower bound. However, the simple two-stage method used here does not establish such a value of β. It should therefore be viewed as a heuristic method motivated by leverage score sampling (Gittens and Mahoney, 2016, Lemma 5).

The difficulty above therefore motivates a more systematic approach. Rather than relying on one small uniform sample to approximate the important subspace immediately, recursive ridge leverage score sampling approximates through a sequence of progressively larger samples. Information obtained from a smaller sample is used to produce score estimates for a larger sample, and the recursion is designed so that the resulting probabilities provide theoretically controlled approximations to ridge leverage scores Musco and Musco (2017). We will see this in the next section.

4.2 Recursive ridge leverage score sampling

The previous section showed the main problems of the approximate leverage method. In this section we introduce Recursive ridge leverage score (RLS) sampling, which addresses this problem differently. Instead of asking one small sample to approximate the important subspace immediately, it estimates scores through a sequence of progressively larger sampled problems.

The algorithm used here is based on Algorithm 2 of Musco and Musco (2017). It first uses ridge leverage scores to provide a regularized measure of the importance of each data point. Then a recursive procedure approximates these scores without forming and decomposing the full kernel matrix.

4.2.1 Ridge leverage scores and effective dimension

Let $K \in \mathbb{R}^{n \times n}$ be a positive semidefinite kernel matrix. For a ridge parameter $\lambda_{\text{RLS}} > 0$, the ridge leverage score of point i is defined as

$$\ell_i^{\lambda_{\text{RLS}}}(K) = [K(K + \lambda_{\text{RLS}} I)^{-1}]_{ii}$$

(Alaoui and Mahoney, 2015).

Ordinary rank-r leverage scores use a fixed number of leading eigenvectors. Ridge leverage scores instead use a cut-off determined by λ_{RLS}. To see this we look at the eigendecomposition of K again, let

$$K = U \Sigma U^T,$$

where

$$\Sigma = \text{diag}(\eta_1, \ldots, \eta_n).$$

Then

$$K(K + \lambda_{\text{RLS}} I)^{-1} = U \Sigma (\Sigma + \lambda_{\text{RLS}} I)^{-1} U^T,$$

and therefore

$$\ell_i^{\lambda_{\text{RLS}}}(K) = \sum_{j=1}^{n} \frac{\eta_j}{\eta_j + \lambda_{\text{RLS}}} U_{ij}^2.$$

The factor $\frac{\eta_j}{\eta_j + \lambda_{\text{RLS}}}$ determines how strongly the j-th spectral direction contributes to the score. If $\eta_j \gg \lambda_{\text{RLS}}$, then this factor is close to 1. If $\eta_j \ll \lambda_{\text{RLS}}$, then it is close to 0. Thus the method controls how much information is kept by controlling λ_{RLS} (Alaoui and Mahoney, 2015; Musco and Musco, 2017).

The sum of the ridge leverage scores is called the *effective dimension*:

$$d_{\text{eff}}^{\lambda_{\text{RLS}}} = \sum_{i=1}^{n} \ell_i^{\lambda_{\text{RLS}}}(K) = \text{tr}(K(K + \lambda_{\text{RLS}} I)^{-1}).$$

Using the eigendecomposition, this becomes

$$d_{\text{eff}}^{\lambda_{\text{RLS}}} = \sum_{j=1}^{n} \frac{\eta_j}{\eta_j + \lambda_{\text{RLS}}}.$$

The effective dimension is not necessarily an integer. It measures the approximate number of spectral directions that remain important at the chosen λ_{RLS}. A smaller λ_{RLS} keeps more directions and gives a larger effective dimension. A larger λ_{RLS} gets rid of more directions and gives a smaller effective dimension (Zhang, 2005; Alaoui and Mahoney, 2015; Musco and Musco, 2017).

Suppose that overestimates of the ridge leverage scores

$$\widetilde{\ell}_i^{\lambda_{\text{RLS}}} \geq \ell_i^{\lambda_{\text{RLS}}}(K)$$

are available. Using overestimates is important because it prevents an important point from being left out due to having a lower sampling probability than its exact ridge leverage score would require. Given these estimates, point i is sampled independently with probability

$$p_i = \min \left\{ 1,\ 16 \widetilde{\ell}_i^{\lambda_{\text{RLS}}} \log \left(\frac{\sum_{j=1}^{n} \widetilde{\ell}_j^{\lambda_{\text{RLS}}}}{\delta} \right) \right\},$$

where δ is a failure probability parameter.

Since every point is sampled independently, the number of selected columns is random rather than fixed in advance. Let

$$Z_i = \begin{cases} 1, & \text{if point } i \text{ is selected,} \\ 0, & \text{otherwise.} \end{cases}$$

Then the total number of selected columns is

$$l = \sum_{i=1}^{n} Z_i.$$

Since point i is selected with probability p_i, we have

$$\mathbb{E}[Z_i] = p_i.$$

Thus

$$\mathbb{E}[l] = \sum_{i=1}^{n} p_i.$$

For exact ridge leverage scores, Musco and Musco (2017) show that

$$\sum_{i=1}^{n} p_i = \mathcal{O}\left(d_{\text{eff}}^{\lambda_{\text{RLS}}} \log\left(\frac{d_{\text{eff}}^{\lambda_{\text{RLS}}}}{\delta} \right) \right).$$

Thus the expected number of selected columns is controlled by the effective dimension. The recursive algorithm later achieves a corresponding high-probability bound on l while avoiding the direct computation of the exact scores.

The remaining difficulty is that the exact ridge leverage scores, and hence the overestimates, cannot be computed cheaply from the full expression much like the exact leverage scores. Algorithm 2 of Musco and Musco (2017) avoids this cost by recursively estimating the scores.

4.2.2 Recursive approximation of the scores

We now describe Algorithm 1 as below from Musco and Musco (2017) where it estimates the scores recursively.

Consider one call of the algorithm on a set of m data points.

1. If the index set is small enough, the recursion stops and returns all current points.
2. Otherwise each current point is included independently with probability 1/2. This produces a uniformly sampled index set containing approximately half of the current points.
3. The same recursive algorithm is applied to this smaller index set. The returned weighted sample gives a compact approximation of the half-sized index set.
4. This approximation is lifted back to the current index set and used to estimate the ridge leverage score of every current point.
5. The current points are sampled according to the resulting probabilities p_i.

The base case in Algorithm 1 is $m \leq 192 \log(1/\delta)$. In this case the method returns the identity matrix, which means that all current points are retained. At every recursive call, the failure-probability parameter is changed from δ to $\delta/3$.

Algorithm 1 RecursiveRLS-Nyström

Input: Data points $x_1, \ldots, x_m \in \mathcal{X}$, kernel function $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, ridge $\lambda > 0$, failure probability $\delta \in (0, 1/32)$

Output: Weighted sampling matrix $S \in \mathbb{R}^{m \times s}$

if $m \leq 192 \log(1/\delta)$ **then**
  return $S := I_{m \times m}$
end if
Let $\overline{\mathcal{S}}$ be a random subset of $\{1, \ldots, m\}$, where each i is included independently with probability $1/2$
▷ *Let* $\overline{X} = \{x_{i_1}, x_{i_2}, \ldots, x_{i_{|\overline{\mathcal{S}}|}}\}$, *where* $i_j \in \overline{\mathcal{S}}$
▷ *Let* $\overline{S} = [e_{i_1}, e_{i_2}, \ldots, e_{i_{|\overline{\mathcal{S}}|}}]$ *be the corresponding sampling matrix*
$\widetilde{S} := \textsc{RecursiveRLS-Nyström}(\overline{X}, K, \lambda, \delta/3)$
$\widehat{S} := \overline{S}\widetilde{S}$
$\widetilde{\ell}_i^{\lambda} := \dfrac{3}{2\lambda}\left(K - K\widehat{S}\left(\widehat{S}^T K \widehat{S} + \lambda I\right)^{-1}\widehat{S}^T K\right)_{ii}$ for every $i \in \{1, \ldots, m\}$
▷ *By Lemma 6 of Musco and Musco (2017), equals* $\frac{3}{2\lambda}(B(B^T\widehat{S}\widehat{S}^T B + \lambda I)^{-1}B^T)_{ii}$. K *denotes the kernel matrix for the points* $\{x_1, \ldots, x_m\}$ *and kernel function* K
$p_i := \min\left\{1,\ 16\widetilde{\ell}_i^{\lambda} \log\left(\dfrac{\sum_{i=1}^m \widetilde{\ell}_i^{\lambda}}{\delta}\right)\right\}$ for each $i \in \{1, \ldots, m\}$
Initialize the weighted sampling matrix S as empty
for $i = 1, \ldots, m$ **do**
  With probability p_i, append the column $\dfrac{1}{\sqrt{p_i}} e_i$ to S
end for
return S

Now suppose that the recursive call on the smaller index set has returned a weighted sampling matrix $\widehat{S}$. Let $C = K\widehat{S}$ and $W = \widehat{S}^T K \widehat{S}$. For point i, let c_i^T denote the ith row of C. The score estimate, i.e. the approximate ridge leverage score, used by the recursive algorithm is

$$\widetilde{\ell}_i^{\lambda_{\text{RLS}}} = \frac{3}{2\lambda_{\text{RLS}}}(K_{ii} - c_i^T(W + \lambda_{\text{RLS}} I)^{-1} c_i)$$

(Musco and Musco, 2017, Algorithm 2).

The term

$$c_i^T(W + \lambda_{\text{RLS}} I)^{-1} c_i$$

measures how much of the ith point is explained by the temporary landmarks returned by the smaller recursive problem. Therefore the residual

$$K_{ii} - c_i^T(W + \lambda_{\text{RLS}} I)^{-1} c_i$$

measures the part that remains unexplained.

If this residual is small then the current point is already represented well by the temporary landmarks and it receives a smaller score. If the residual is large then the point contains information that is not well represented and thus receives a larger probability. This is the essential adaptive part of the recursive RLS.

The factor $3/2$ is an oversampling factor and its purpose is to overestimate the scores. Overestimates are important because underestimating an important point could cause it to be sampled with too little probability as we mentioned earlier.

When point i is selected, this algorithm uses the weighted column $\frac{1}{\sqrt{p_i}} e_i$ in the sampling matrix. These weights are necessary while the sample is used inside later recursive calls

but they do not change the final Nyström approximation associated with the final indices. Therefore after the recursion has returned the final landmark set, the factorization solver can use the selected indices to construct $C = K_{:,I}$ and $W = K_{I,I}$.

4.2.3 Approximation guarantee

The recursive construction is not merely a heuristic as the approximate leverage method from the previous section. The recursion and oversampling factors are designed to produce controlled overestimates of the exact ridge leverage scores. We present the main theoretical guarantee of Musco and Musco (2017) in the following theorem.

Theorem 4.1 (Recursive RLS Nyström; Musco and Musco, 2017, Theorem 7)**.** *Let $K \in \mathbb{R}^{n\times n}$ be positive semidefinite, let $\lambda_{\mathrm{RLS}} > 0$ and $\delta \in (0, 1/32)$. Suppose the weighted sampling matrix $S \in \mathbb{R}^{n\times l}$ is produced by Algorithm 1. Then with probability at least $1 - 3\delta$, the number of selected columns satisfies*

$$l = \mathcal{O}\left(d_{\mathrm{eff}}^{\lambda_{\mathrm{RLS}}} \log\left(\frac{d_{\mathrm{eff}}^{\lambda_{\mathrm{RLS}}}}{\delta}\right)\right),$$

and the Nyström approximation

$$\widetilde{K} = KS(S^T KS)^{\dagger} S^T K$$

satisfies

$$\widetilde{K} \preceq K \preceq \widetilde{K} + \lambda_{\mathrm{RLS}} I.$$

The algorithm uses $\mathcal{O}(nl)$ kernel evaluations and $\mathcal{O}(nl^2)$ additional computation.

The first inequality $\widetilde{K} \preceq K$ agrees with the general Nyström result from Chapter 3 as the Nyström approximation approximates the kernel matrix from below.

The second inequality gives

$$0 \preceq K - \widetilde{K} \preceq \lambda_{\mathrm{RLS}} I.$$

It follows immediately that

$$\|K - \widetilde{K}\|_2 \leq \lambda_{\mathrm{RLS}}.$$

Thus λ_{RLS} has a natural theoretical meaning. A smaller value preserves more of the spectrum and generally requires more landmarks. A larger value allows more spectral information to be discarded and requires fewer landmarks.

The recursion then approximates at each level by retaining approximately half of the current points, and the smaller problem is used to estimate the scores for the larger problem. If the final sample contains l landmarks, the cost across all levels has the geometric series

$$\mathcal{O}\left(nl^2\right) + \mathcal{O}\left(\frac{n}{2}l^2\right) + \mathcal{O}\left(\frac{n}{4}l^2\right) + \cdots.$$

Thus the total cost remains $\mathcal{O}(nl^2)$. The method therefore avoids computing the ridge leverage scores from forming the full $n \times n$ kernel matrix.

4.2.4 Relation between the sampling parameter and KRR regularization

Two different parameters appear in the recursive RLS experiment. The KRR regularization parameter λ_{KRR} is used in the prediction system

$$(\widetilde{K} + \lambda_{\mathrm{KRR}} I)\widetilde{\alpha} = y.$$

Thus it is used after uniform, approximate leverage and recursive RLS sampling. Therefore, all three methods solve the same approximate KRR problem once their landmark columns have been selected.

Recursive RLS additionally uses λ_{RLS} during landmark selection. It appears in the ridge leverage scores

$$\ell_i^{\lambda_{\mathrm{RLS}}}(K) = [K(K + \lambda_{\mathrm{RLS}} I)^{-1}]_{ii}.$$

and controls the decision on which directions of the kernel matrix are considered important. It does not replace λ_{KRR} in the final solve (Alaoui and Mahoney, 2015; Musco and Musco, 2017).

The numerical values of these parameters are very different in the experiment. The best KRR regularization found in Chapter 2 is approximately $\lambda_{\mathrm{KRR}} = 0.01$, whereas recursive RLS seems to scale with the total training size and requires values of λ_{RLS} around 10^3 or larger to return a reasonable number of landmarks. This large difference is most likely caused by the scale of the RBF kernel matrix.

For the RBF kernel $K_{ii} = 1$ and hence $\mathrm{tr}(K) = n$. Although the eigenvalues decay, Figure 2.1 shows their values normalised by the largest eigenvalue and therefore only describes the relative spectrum. The same normalised decay pattern can remain as n increases while the unnormalised eigenvalues increase in absolute scale. The total eigenvalue mass therefore increases with the training size. We take a look at the effective dimension

$$d_{\mathrm{eff}}^{\lambda_{\mathrm{RLS}}} = \sum_{j=1}^{n} \frac{\eta_j}{\eta_j + \lambda_{\mathrm{RLS}}}.$$

If λ_{RLS} is too small relative to the eigenvalues of K, the effective dimension becomes large. Many sampling probabilities then become close to one and the method may select a very large portion of the training set. The large value of λ_{RLS} is therefore needed to keep the column selection count reasonable.

This interpretation is also consistent with the empirical evaluation of Musco and Musco (2017). They consider selecting a prescribed landmark budget. In that setting, the recursive procedure determines λ_{RLS} from the spectrum of the unnormalised kernel matrix. Since this depends on the tail eigenvalue mass, it can be numerically large when the kernel eigenvalues grow with the sample size. Thus the large values of λ_{RLS} observed in this project are consistent with the scale used by their sampling framework, even though they are much larger than λ_{KRR}. Therefore their empirical approximation results also appear to evaluate whether the approximation works well in practice rather than directly demonstrating the approximation guarantee.

The theory connects λ_{RLS} and λ_{KRR} when a formal guarantee on the approximate KRR estimator is required (Alaoui and Mahoney, 2015; Musco and Musco, 2017). The independently chosen values used in this project do not satisfy that theoretical coupling. Therefore the guarantee is not claimed for the recursive experiments. Instead the method is evaluated empirically by measuring the prediction error, kernel approximation error and runtime produced by the selected landmarks.

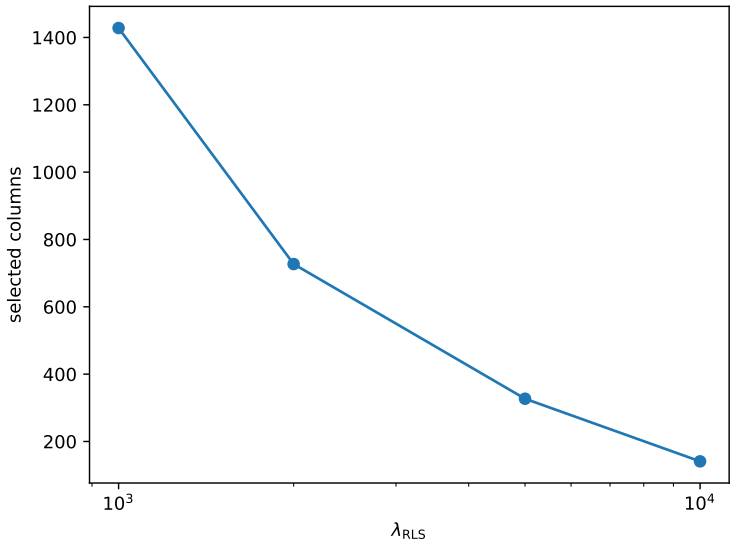


Figure 4.6: Number of selected recursive RLS columns against λ_{RLS}.

Figure 4.6 shows that increasing λ_{RLS} decreases the number of selected columns. A larger value reduces the effective dimension and therefore lowers the sampling probabilities. In contrast, small values cause the algorithm to return a large number of columns.

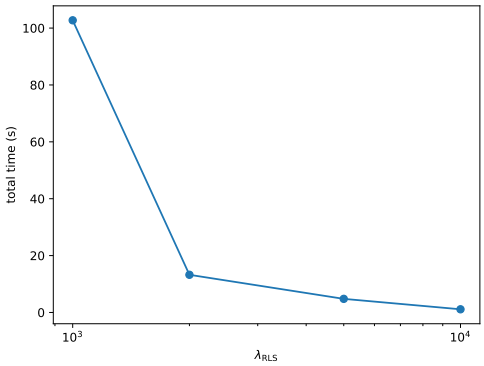


Figure 4.7: Recursive RLS total time against λ_{RLS}.

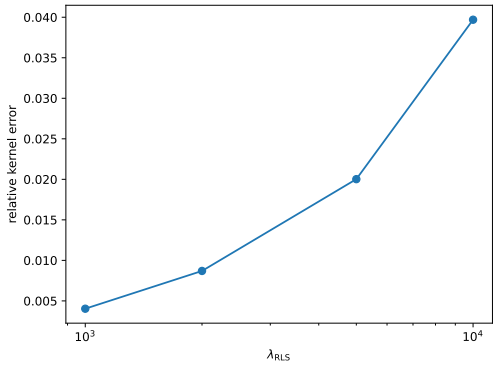


Figure 4.8: Recursive RLS kernel error against λ_{RLS}.

Figures 4.7 and 4.8 show the resulting trade off. Smaller values of λ_{RLS} select more landmarks and produce a more accurate kernel approximation, but the sampling and factorization costs increase. Larger values select fewer landmarks and reduce runtime, but the kernel approximation error increases.

This completes the discussion on informed column sampling methods. The next section discusses the experimental setup and compares recursive RLS with uniform and approximate leverage sampling using a variety of metrics.

5 Experimental results and analysis

5.1 Final experimental setup and aggregation

The main experimental question is whether Nyström approximation can keep the prediction error close to full KRR while reducing runtime. The Superconductivity dataset remains the main experimental example, while two additional regression datasets are used in Section 5.6 to determine whether the observed behaviour is specific to one kernel matrix.

Dataset	Available observations	Input features	Target
Superconductivity	21 263	81	critical temperature
CASP protein structure	45 730	9	RMSD
CT Slice Localization	53 500	384	relative axial location

Table 5.1: Regression datasets used in the final experiments.

All three datasets were obtained from the UCI Machine Learning Repository (Hamidieh, 2018; Rana, 2013; Graf et al., 2011). The patient identifier in the CT dataset was excluded from the input features.

For every dataset, we used 19 000 observations for training and 1 000 for testing. The train/test split was generated once and kept fixed throughout all five repetitions. The input features were standardized using the mean and standard deviation of the training data. The Gaussian RBF kernel

$$k(x, z) = \exp(-\gamma \|x - z\|_2^2)$$

was used with $\gamma = \frac{1}{d}$, where d is the number of input features. The KRR regularization parameter was fixed at $\lambda_{\mathrm{KRR}} = 0.01$, which gave a test MSE very close to or at the minimum of a preliminary parameter sweep for all three datasets.

The methods compared were full KRR, uniform Nyström sampling, approximate leverage score sampling and recursive ridge leverage score (RLS) sampling. For uniform and approximate leverage sampling, the tested column counts were

$$l \in \{50, 100, 200, 400, 800, 1600\}.$$

The approximate leverage sampling used an initial uniform sample of

$$l_0 = \min\{100, \max(1, \lfloor l/2 \rfloor)\}.$$

For recursive RLS, the tested parameters were

$$\lambda_{\mathrm{RLS}} \in \{1000, 2000, 5000, 10000, 20000\}, \qquad \delta = 0.1.$$

The actual number of returned columns is random for recursive RLS because the method samples each point with probability p_i.

Each randomized method was run five times. The train/test split and all deterministic parameters were fixed while the randomized column selection was independently redrawn. For every metric, we computed the empirical mean, sample variance and sample standard deviation. The figures display the empirical mean with bars of one sample standard deviation. Recursive RLS is plotted at its mean selected column count and therefore has horizontal bars as well as vertical bars.

The full KRR solution does not change between repetitions because the split and parameters are fixed. Its dense solve and test prediction runtime was nevertheless repeated

five times to obtain a stable mean solve time. The full KRR total reference time was obtained by adding the construction time of the full training and testing kernel matrices to the mean solve time.

The first metric is the relative kernel approximation error

$$\frac{\|K - \tilde{K}\|_F}{\|K\|_F}.$$

This measures how well the matrix K itself is approximated. The second metric is the relative coefficient error

$$\frac{\|\alpha - \tilde{\alpha}\|_2}{\|\alpha\|_2},$$

where α is the full KRR solution and $\tilde{\alpha}$ is the approximate Nyström solution. The third metric and the most important metric is test prediction error, measured by MSE and RMSE. The RMSE is especially useful in the final discussion because it is in the original units of the target variable (Cortes et al., 2010).

To compare prediction error directly with the full KRR baseline, the relative excess test RMSE is defined as

$$\frac{\text{RMSE}_{\text{approx}} - \text{RMSE}_{\text{full}}}{\text{RMSE}_{\text{full}}}.$$

A value of zero means that the approximation matches the full KRR test RMSE. A value of 0.1 means that the approximation has about 10% larger RMSE than full KRR.

Runtime is split into sampling time and solve time. Sampling time measures the cost of choosing columns. Solve time includes constructing C and W, computing the factorized Nyström approximation, solving for $\tilde{\alpha}$ and producing the test predictions. Total time is the sum of these two. This distinction matters because, as we will see, the sampling step is not the main bottleneck once l is moderately large. We now talk about the core results of this project.

5.2 Prediction error against sampled columns

We first look at prediction error as a function of the number of sampled columns. This is the most direct way to see whether Nyström prediction moves toward full KRR as l increases.

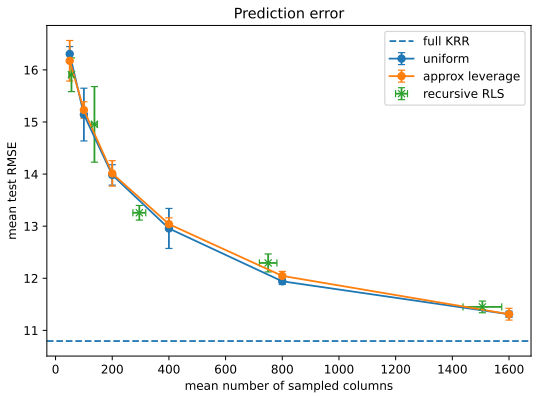


Figure 5.1: Mean test RMSE against mean l.

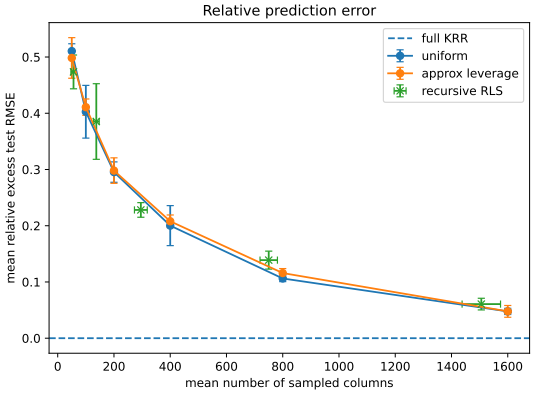


Figure 5.2: Mean relative excess test RMSE against mean l.

Figure 5.1 shows that the mean test RMSE decreases as more columns are sampled. This is as expected by theory (Cortes et al., 2010; Bach, 2013), $\tilde{K}$ captures more information

of K with more columns, and the approximate KRR prediction becomes closer to the full KRR prediction. For the current experiments, the Nyström curves approach this line, but they do not reach it for the tested values of l. This is a limitation of the tested landmark range but we see in the relative excess RMSE plot that the gap is small for the largest l in figure 5.2. At $l \approx 1600$, all methods have a mean test RMSE of approximately 11.1, compared with around 10.9 for full KRR.

Uniform and approximate leverage sampling are very close. Approximate leverage is not consistently better even though it is more informed than uniform sampling theoretically. As we have seen before this may be due to the large amount of noise in the approximate scores compared to exact scores. Recursive RLS follows the same general trend, but its points are not on the same fixed-l grid because the method chooses a random number of columns through its sampling probabilities.

The prediction error bars are small compared to the improvement achieved by increasing l. Therefore the decreasing error pattern is stable across the five column selections and is not caused by one particularly good or bad sample. The error bars and error curves of all methods are also similar for much of the tested range, so the repeated experiment does not show a consistent prediction advantage for approximate leverage sampling or recursive RLS sampling.

Figure 5.2 gives the same prediction result relative to the full KRR baseline. It makes the practical interpretation clearer as small values of l are much worse than full KRR, while larger values reduce the gap. For uniform sampling, the mean relative excess RMSE decreases from approximately 51% at $l = 50$ to 20% at $l = 400$, 11% at $l = 800$ and 5% at $l = 1600$. Approximate leverage sampling and recursive RLS sampling follow almost the same curve. The relative excess RMSE for the largest fixed column count is small compared with the low columns cases. Thus the important question is not only whether increasing l improves prediction, but also if the decrease in test error is worth the runtime.

5.3 Runtime and prediction trade off

In this section we investigate the runtime and prediction trade off. This is the central practical objective and the next figures are therefore the main results of the project.

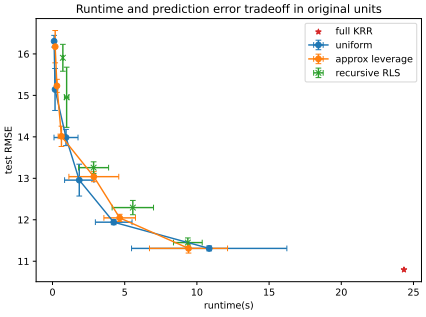


Figure 5.3: Mean runtime and test RMSE trade off.

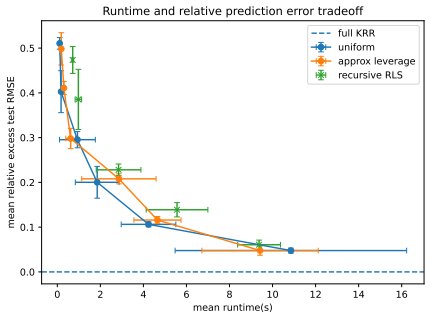


Figure 5.4: Mean runtime and relative excess test RMSE compared with full KRR.

Figure 5.3 is the main prediction against runtime figure. It shows that all Nyström methods improve as runtime increases, but the best prediction errors occur only at the larger tested runtimes. The full KRR reference point has the lowest RMSE. The Nyström methods approach it, but the runtime advantage becomes smaller when there is a high number of columns selected. Although the runtime advantage narrows as l increases, the largest tested Nyström points still use less than half of the full KRR total reference time.

Figure 5.4 gives a clear normalized view of the same result. A value of zero means

full KRR prediction accuracy. Values above zero measure how much worse the Nyström method is in RMSE compared to full KRR. The plot shows a clear saturation pattern where the first increases in runtime give large prediction improvements, but the last part of the gap to full KRR is expensive. The elbow of the curve is around 0.1 relative excess RMSE and approximately 4–5 seconds runtime, which means that the Nyström methods have about 10% larger RMSE than full KRR using around one fifth of the runtime. In this experiment, this provides a reasonable trade off even though it still accepts a noticeable prediction loss. Increasing the fixed column count to $l = 1600$ reduces the relative excess RMSE to approximately 5%, but raises the mean runtime to approximately 11 seconds for uniform sampling and 9.5 seconds for approximate leverage sampling. Thus roughly doubling the runtime only halves the remaining excess RMSE from approximately 10% to 5%. This quantifies the diminishing returns near the full KRR solution. Of course, this is only one main dataset and one set of algorithms. Other datasets may have different trade offs as investigated in Section 5.6.

5.4 Runtime analysis and solver behaviour

If we compare the previous plots on runtime and columns, we see that the runtime increases quickly with l. Specifically, at $l \approx 1500$–1600, the time has already reached around 40% of the mean full KRR total reference time. The question is why the runtime behaves this way. The next two figures show the sampling time and total time against l. The possible reason is that the sampling step itself is not the main bottleneck. Once the columns have been selected, all methods pass those columns to the same factorization solver.

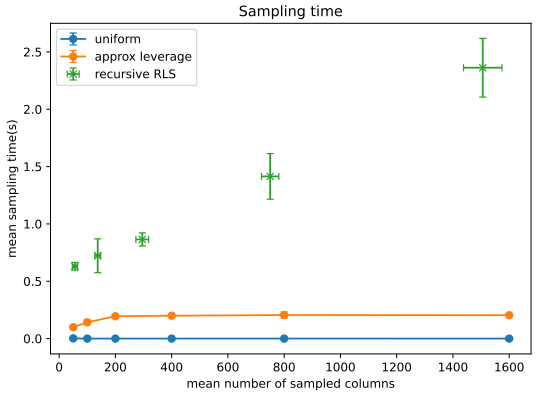


Figure 5.5: Mean sampling time against mean l.

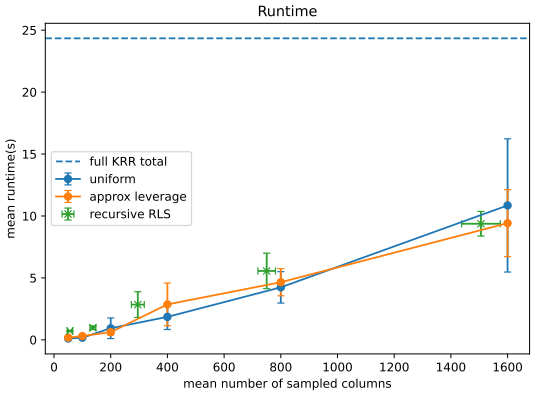


Figure 5.6: Mean total runtime against mean l.

Figure 5.5 shows that uniform sampling is essentially free, and approximate leverage sampling only adds a small cost. Recursive RLS has a larger sampling cost which looks approximately linear with the number of columns, but this is not the main bottleneck at large column counts compared to the solve time since all methods have similar total times at large l. Figure 5.6 shows the total runtime. It rises quickly as l increases and becomes a substantial fraction of the full KRR total reference time at the largest fixed column count. This shows that optimizing the sampling step alone may not be enough to reduce the overall runtime. However, at the smaller recursive samples, the recursive sampling step can still form a substantial fraction of the total runtime while uniform sampling remains essentially free.

For example, with $\lambda_{\mathrm{RLS}} = 1000$, recursive RLS selected a mean of 1506 columns with a standard deviation of approximately 68 columns. Its mean sampling time was approximately 2.36 seconds, while the common solve after column sampling took approximately

7.01 seconds, giving a mean total runtime of approximately 9.37 seconds. The full KRR total reference time was approximately 24.34 seconds, consisting of approximately 6.37 seconds for full kernel construction and 17.97 seconds for the dense solve and test prediction. Recursive Nyström was therefore approximately 2.6 times faster overall at this point, while having approximately 6% relative excess test RMSE. However, the common solve still formed about three quarters of its Nyström runtime, showing that this solver becomes the main Nyström bottleneck at large l. Therefore, the solver is efficient when $l \ll n$, which means that Nyström approximation helps most if the sampling method keeps l genuinely small.

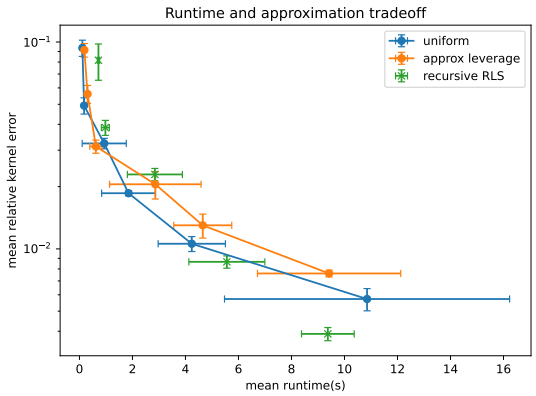


Figure 5.7: Mean runtime and kernel approximation error.

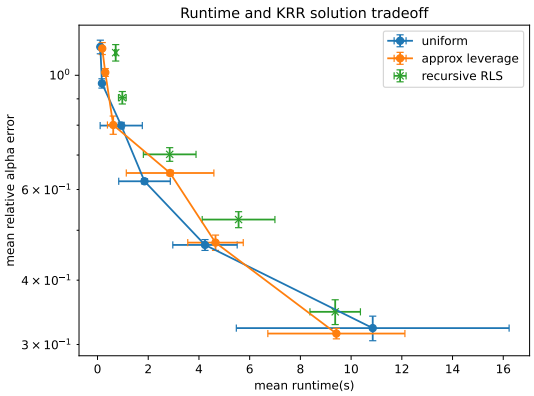


Figure 5.8: Mean runtime and coefficient error.

Figures 5.7 and 5.8 show the same general trade off at the matrix and coefficient levels, runtime increases generally give better approximations. Recursive RLS seems to achieve a smaller kernel approximation error at a similar runtime, but this does not produce the same clear advantage in coefficient error or test RMSE. Therefore a slightly better approximation of K does not automatically give a better prediction runtime trade off.

5.5 Secondary results and residual diagnostics

The next figures check whether the approximation and the solver behave consistently. They are not the central results but they can show whether large errors come from the solver failing or from the approximation losing important information.

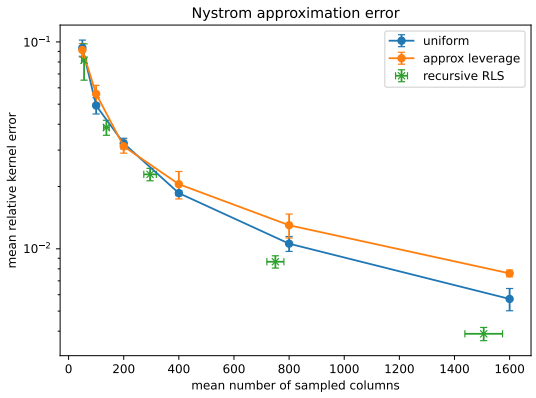


Figure 5.9: Mean relative kernel error.

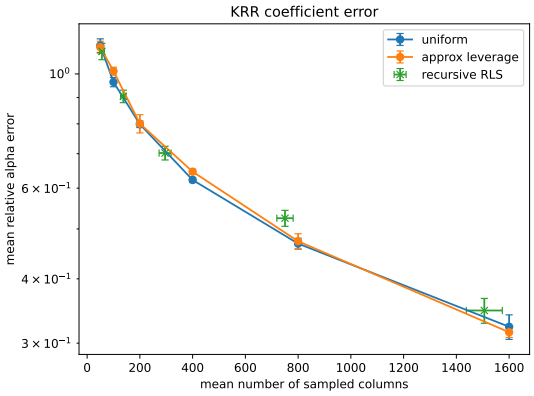


Figure 5.10: Mean relative KRR coefficient error.

Figure 5.9 shows that the direct matrix approximation error decreases with l. Recursive RLS can have very small kernel error when it selects enough columns. At a mean of 1506 columns, its mean relative kernel error is approximately 0.0039, compared with approximately 0.0057 for uniform sampling and 0.0076 for approximate leverage sampling at $l = 1600$. Figure 5.10 shows similar behavior, however, recursive RLS does not have the smallest coefficient error at these large column counts. Thus the ordering of the methods by kernel approximation error does not necessarily match their ordering by coefficient error.

The residual diagnostics below come from a representative single run rather than the five run averages. They are kept because their purpose is to validate the solver rather than to compare the expected performance of sampling methods.

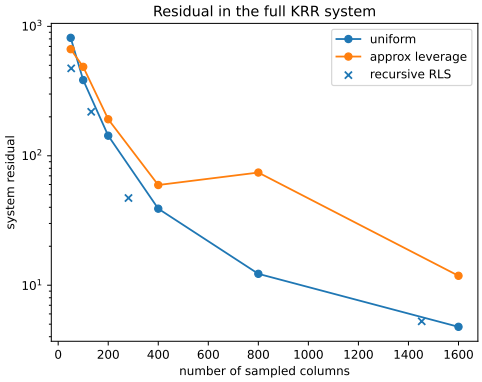


Figure 5.11: Residual in the full KRR system for a representative run.

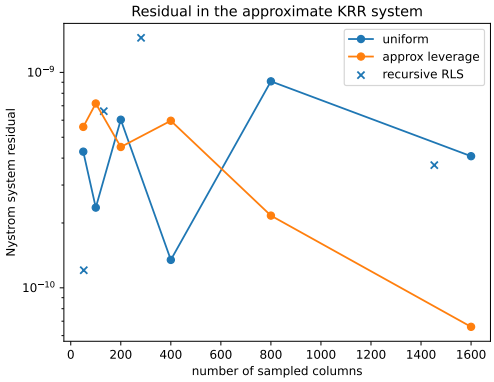


Figure 5.12: Residual in the approximate Nyström system for a representative run.

Figure 5.11 inserts the approximate solution into the original full system and measures

$$\frac{\|(K + \lambda_{\mathrm{KRR}} I)\tilde{\alpha} - y\|_2}{\|y\|_2}.$$

This residual decreases as the approximation improves, but it is not close to zero. That is as expected since $\tilde{\alpha}$ solves the approximate system instead of the full system. Figure 5.12 measures the residual in the approximate system

$$\frac{\|(\tilde{K} + \lambda_{\mathrm{KRR}} I)\tilde{\alpha} - y\|_2}{\|y\|_2}.$$

These residuals are very small. Therefore the solver is doing what it should and it solves the approximate Nyström system accurately. The remaining error comes from replacing K by $\tilde{K}$ instead of a numerical failure of the solve.

5.6 Repeated experiments on additional datasets

To determine whether the behaviour observed for Superconductivity is specific to one kernel matrix, the same five repetition experiment was carried out on the CASP protein structure dataset and the CT Slice Localization dataset (Rana, 2013; Graf et al., 2011). We repeated the exact same set up as the main dataset and obtained the following.

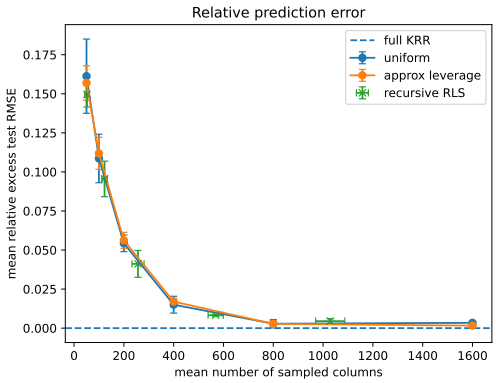


Figure 5.13: Mean relative excess test RMSE for CASP.

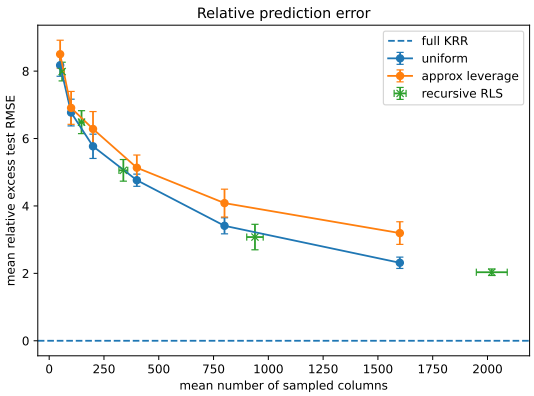


Figure 5.14: Mean relative excess test RMSE for CT Slice Localization.

The two datasets show the same general pattern as the main dataset, increasing the number of landmarks generally improves prediction accuracy. However the methods remain very close on CASP while clearer differences seem to be present at larger column counts on CT Slice Localization. Moreover, the rate at which the predictions of all methods approach full KRR differs substantially between the datasets.

Figure 5.13 shows that CASP converges much faster than Superconductivity. Uniform sampling has approximately 1.5% relative excess RMSE at $l = 400$ and approximately 0.3% at $l = 800$. The other two sampling methods give almost the same result. Therefore CASP reaches the full KRR prediction region with substantially fewer columns than Superconductivity.

Figure 5.14 shows the opposite case. CT Slice Localization improves as l increases, but it does not approach full KRR over the tested range. At $l = 1600$, uniform sampling still has approximately 231% relative excess RMSE and approximate leverage sampling approximately 320%. Recursive RLS has approximately 203% relative excess RMSE even when it returns a mean of approximately 2020 columns. It seems that we would need a lot more columns to reduce the error further. Superconductivity therefore lies between the rapidly converging CASP case and the much more slowly converging CT Slice Localization case.

The previous figures compare prediction accuracy at a given number of sampled columns. However, to compare the practical trade off directly, Figure 5.15 and 5.16 plot the relative excess test RMSE against the mean total runtime of each Nyström method.

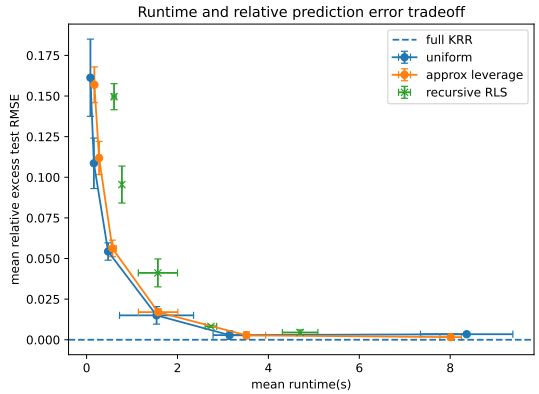


Figure 5.15: Mean runtime and relative excess test RMSE for CASP.

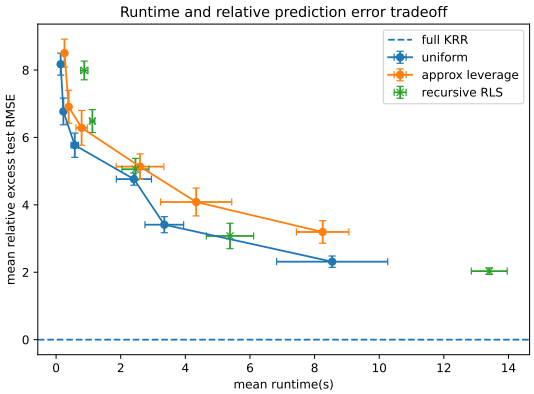


Figure 5.16: Mean runtime and relative excess test RMSE for CT Slice Localization.

Uniform and approximate leverage sampling remain close on CASP with recursive RLS slightly worse at low column count. However, approximate leverage sampling performs worse than uniform sampling at the larger landmark counts on CT Slice Localization. Recursive RLS does not provide a consistent prediction advantage across the datasets. These results strengthen the earlier conclusion that a more informed sampling strategy does not automatically produce a better prediction runtime trade off.

To investigate why the three datasets require different numbers of columns, the eigenvalues of fixed 2000×2000 kernel submatrices were compared. This was repeated across different ns which all gave the same pattern. The figure below shows a controlled spectral comparison without requiring the complete eigendecomposition of each 19000×19000 kernel matrix.

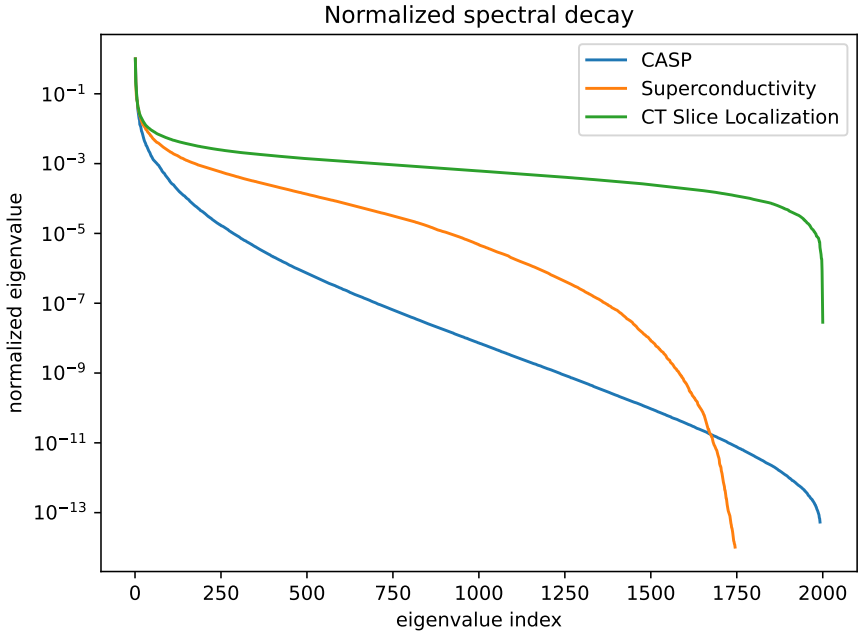


Figure 5.17: Normalized eigenvalue decay for 2000×2000 kernel submatrices.

Figure 5.17 shows that the CASP eigenvalues decay the fastest, the Superconductivity eigenvalues have an intermediate decay, and the CT Slice Localization eigenvalues decay slowest.

The relative spectral decay is consistent with the prediction curves. The faster decay for CASP is consistent with its requiring relatively few landmarks, while the slower decay for CT Slice Localization is consistent with its much slower approach to full KRR. Since the eigenvalues are normalized, this figure gives a qualitative spectral explanation rather than directly showing how many directions are important at $\lambda_{\mathrm{KRR}} = 0.01$.

This does not mean that spectral decay is the only reason for the differences between the datasets. However, this gives a reasonable explanation for why the required landmark count is dataset dependent. The additional experiments therefore show that the improvement with increasing l is stable across datasets, but the number of columns required to approach full KRR can vary significantly across different kernel matrices.

6 Conclusions

This project investigated randomized Nyström approximation for kernel ridge regression. The main objective was to determine how the column sampling strategy and the number of sampled columns affect prediction accuracy, approximation quality and computational cost. Uniform sampling, approximate leverage score sampling and recursive ridge leverage score sampling were compared against a full KRR baseline. The final experiment used five independent column selections on the Superconductivity, CASP protein structure and CT Slice Localization datasets, with the same training size and a fixed train/test split for each dataset.

The first research question asked which Nyström column sampling strategies give good performance in terms of prediction accuracy and computational cost. In the experiments, uniform sampling was the strongest practical baseline. It has essentially no sampling cost and its prediction results were very close to those of the more informed methods. Approximate leverage sampling did not consistently improve on uniform sampling. The diagnostic experiments showed that its estimated scores can be noisy and depend strongly on the quality of the initial uniform sample. Therefore the additional score estimation step did not reliably produce better landmark sets.

Recursive RLS provides a more systematic and theoretically motivated way of estimating column importance (Musco and Musco, 2017). It could produce a small kernel approximation error when sufficiently many columns were selected, but it also had the largest sampling cost and did not give a clear improvement in the final prediction-runtime trade off. In particular, the smaller kernel approximation error obtained by recursive RLS at large column counts did not imply the smallest coefficient error or test RMSE. Its sampling parameter λ_{RLS} also required careful selection because it controls the effective dimension and therefore the number of returned landmarks (Alaoui and Mahoney, 2015; Musco and Musco, 2017). The large values needed in the experiment are related to the scale of the RBF kernel matrix. Since these values were selected independently from λ_{KRR}, the recursive results should be interpreted empirically rather than as a direct verification of the theoretical guarantee.

The second research question asked how large the sampled column count l needs to be and how this varies across datasets, sampling strategies and for recursive RLS, λ_{RLS}. No single value of l is sufficient for every required level of accuracy or every dataset. Small landmark sets gave substantial runtime reductions, but their prediction errors were noticeably larger than full KRR. Increasing l generally reduced the kernel error, coefficient error and test RMSE. The effect of increasing l differed significantly between datasets. CASP approached full KRR with relatively few columns, Superconductivity approached it more gradually, and CT Slice Localization remained far from full KRR over the tested range. Within each dataset, the required value of l was not consistently reduced by using the more informed sampling methods. The kernel matrix and its spectral decay seemed to have a stronger effect on the required landmark count than the choice between the three tested sampling strategies. This is consistent with theoretical analyses that relate the required approximation rank to the spectrum and effective dimension of the kernel matrix (Bach, 2013; Rudi et al., 2015).

The third research question asked what the accuracy/cost trade off looks like as l increases and whether a reasonable practical approach can be recommended. The main Superconductivity experiment showed a clear saturation pattern. The first increases in l produced large improvements in prediction accuracy for a relatively small increase in runtime. Closing the final part of the gap to full KRR was much more expensive. In the main experiment, a point around ten percent relative excess test RMSE required approximately

4 to 5 seconds, or roughly one fifth of the full KRR total reference time. Reducing the gap to approximately 5% required around 10 seconds, corresponding to approximately 40% of the full KRR total reference time. This provides a reasonable compromise when a small loss of prediction accuracy is acceptable. The additional datasets show that the position of this trade off is not universal as one would expect. CASP gave a substantially more favourable trade off, while CT Slice Localization was not close to the full KRR prediction region within the tested range.

We also saw that column selection was not the dominant cost at large l. Once the landmarks had been chosen, every sampling method used the same factorized solver with dominant cost $\mathcal{O}(nl^2 + l^3)$ (Gittens and Mahoney, 2016). Therefore even a theoretically better sampling method is only useful in practice if it reaches the required prediction quality with a substantially smaller column selection set. When l becomes a significant fraction of n, the common solve dominates and the differences between the sampling costs become relatively unimportant.

Overall, the practical recommendation from this project is to begin with uniform sampling. It is simple, cheap and was competitive throughout the experiments. A more informed sampling method is worth it only when the kernel matrix has sufficiently nonuniform spread of directions in columns and the approximation method can use this structure to achieve the same prediction quality with less landmarks. This agrees with previous results showing that leverage-based sampling offers less advantage when the leverage scores are relatively uniform (Gittens and Mahoney, 2016; Musco and Musco, 2017). In the experiments we did not observe this advantage consistently.

6.1 Limitations and future work

The final experiments used three datasets which gives a stronger comparison than a single dataset experiment but it is still too small to establish a universal ranking of the sampling methods. In addition, only one fixed train/test split was used for each dataset. The five repetitions therefore measure variation caused by randomized column selection, but they do not measure variation caused by changing the training and test observations. Lastly the amount of training data we used were conservative in consideration of computing power limitations. Future experiments should repeat the comparison over several independent train/test splits, a wider collection of datasets and a much larger training size.

The same RBF kernel and the same kernel rule $\gamma = \frac{1}{d}$ were used for all three datasets. This gives a controlled comparison, but it does not mean that the KRR model was optimally tuned for every dataset. In particular, the poor approximation results on CT Slice Localization may partly be the result of a kernel configuration that retained many important spectral directions. Future work should tune γ, λ_{KRR} and l in combination for each dataset and then investigate relative changes.

A further limitation is the runtime measurements. Runtime depends on the experimental machine and its current workload. Background processes and other system level effects can change the measured runtime significantly even when the data and algorithms are unchanged. The reported runtimes and their corresponding analysis should therefore be interpreted as relative results rather than universal results. The asymptotic growth with l and the observation that the common factorized solve becomes dominant are much more transferable than the exact runtime in seconds. A better benchmark experiment could be run on an idle and advanced machine, include warm up runs (first runs being slow was observed as outliers), repeat every measurement several times and ideally report the hardware and software configuration.

The recursive sampling parameter λ_{RLS} was selected separately from λ_{KRR} and chosen to produce computationally manageable column counts. Using a value near $\lambda_{\mathrm{KRR}} = 0.01$ would retain a very large effective dimension and cause recursive RLS to select a large

fraction of the training set (most of the time the entire set). Consequently the recursive experiments provide an empirical comparison of the algorithm at different returned column counts, but they do not directly verify the theoretical KRR guarantee associated with a particular coupling between regularization and column sampling. Future work should study this coupling in more detail including the effect of kernel selection and training data size.

Most importantly, the results show that improving sampling strategy alone does not resolve the runtime bottleneck. Once a large value of l is required, the direct factorized solver has dominant cost $\mathcal{O}(nl^2 + l^3)$ regardless of how the columns were selected. Future work could investigate iterative Nyström solvers based on preconditioned conjugate gradients rather than only the direct factorized solver we used in this project (Rudi et al., 2017).

Finally, the experiments showed that a smaller Frobenius kernel error did not necessarily lead to a smaller coefficient error or test RMSE, suggesting that prediction-aware landmark selection may be a useful direction for future research (Cortes et al., 2010; Alaoui and Mahoney, 2015). Although prediction error was evaluated in this project with the two nonuniform sampling strategies using information from the kernel matrix, none of the three methods used validation prediction error when selecting landmarks. Future work could use a three-way training/validation/test split, where the validation set guides the choice of l while the test set remains reserved for final evaluation. Such a rule should stop increasing l when the additional validation error reduction no longer justifies the additional runtime.

Bibliography

Alaoui, A. and Mahoney, M. W. (2015). Fast randomized kernel ridge regression with statistical guarantees. *Advances in neural information processing systems*, 28.

Aronszajn, N. (1950). Theory of reproducing kernels. *Transactions of the American mathematical society*, 68(3):337–404.

Bach, F. (2013). Sharp analysis of low-rank kernel matrix approximations. In *Conference on learning theory*, pages 185–209. PMLR.

Calandriello, D., Lazaric, A., and Valko, M. (2017). Distributed adaptive sampling for kernel matrix approximation. In *Artificial Intelligence and Statistics*, pages 1421–1429. PMLR.

Cortes, C., Mohri, M., and Talwalkar, A. (2010). On the impact of kernel approximation on learning accuracy. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 113–120. JMLR Workshop and Conference Proceedings.

Drineas, P., Magdon-Ismail, M., Mahoney, M. W., and Woodruff, D. P. (2012). Fast approximation of matrix coherence and statistical leverage. *The Journal of Machine Learning Research*, 13(1):3475–3506.

Drineas, P., Mahoney, M. W., and Cristianini, N. (2005). On the nyström method for approximating a gram matrix for improved kernel-based learning. *journal of machine learning research*, 6(12).

Eckart, C. and Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218.

Gittens, A. and Mahoney, M. (2013). Revisiting the nystrom method for improved large-scale machine learning. In *International Conference on Machine Learning*, pages 567–575. PMLR.

Gittens, A. and Mahoney, M. W. (2016). Revisiting the nyström method for improved large-scale machine learning. *Journal of Machine Learning Research*, 17:1–65.

Graf, F., Kriegel, H.-P., Schubert, M., Poelsterl, S., and Cavallaro, A. (2011). Relative location of CT slices on axial axis. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5CP6G.

Halko, N., Martinsson, P.-G., and Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM review*, 53(2):217–288.

Hamidieh, K. (2018). Superconductivty Data. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C53P47.

Hofmann, T., Schölkopf, B., and Smola, A. J. (2008). Kernel methods in machine learning.

Kimeldorf, G. and Wahba, G. (1971). Some results on tchebycheffian spline functions. *Journal of mathematical analysis and applications*, 33(1):82–95.

Kumar, S., Mohri, M., and Talwalkar, A. (2012). Sampling methods for the nyström method. *The Journal of Machine Learning Research*, 13(1):981–1006.

Martinsson, P.-G. and Tropp, J. A. (2020). Randomized numerical linear algebra: Foundations and algorithms. *Acta Numerica*, 29:403–572.

Musco, C. and Musco, C. (2017). Recursive sampling for the nystrom method. *Advances in neural information processing systems*, 30.

Nyström, E. J. (1930). Über die praktische auflösung von integralgleichungen mit anwendungen auf randwertaufgaben.

Penrose, R. (1955). A generalized inverse for matrices. In *Mathematical proceedings of the Cambridge philosophical society*, volume 51, pages 406–413. Cambridge University Press.

Rana, P. (2013). Physicochemical Properties of Protein Tertiary Structure. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5QW3H.

Rudi, A., Camoriano, R., and Rosasco, L. (2015). Less is more: Nyström computational regularization. *Advances in neural information processing systems*, 28.

Rudi, A., Carratino, L., and Rosasco, L. (2017). Falkon: An optimal large scale kernel method. *Advances in neural information processing systems*, 30.

Saunders, C., Gammerman, A., and Vovk, V. (1998). Ridge regression learning algorithm in dual variables.

Schölkopf, B., Herbrich, R., and Smola, A. J. (2001). A generalized representer theorem. In *International conference on computational learning theory*, pages 416–426. Springer.

Trefethen, L. and Bau, D. (1997). *Numerical Linear Algebra*. Other Titles in Applied Mathematics. Society for Industrial and Applied Mathematics (SIAM, 3600 Market Street, Floor 6, Philadelphia, PA 19104).

Williams, C. and Seeger, M. (2000). Using the nyström method to speed up kernel machines. *Advances in neural information processing systems*, 13.

Zhang, T. (2005). Learning bounds for kernel regression using effective data dimensionality. *Neural computation*, 17(9):2077–2098.

Zhu, R. (2023). Reproducing kernel hilbert space. Chapter 15 of *Statistical Machine Learning with R*.